# Автентифікація як сервіс безпеки додатків
# AAA

**2 етапа:**

**1. Ідентифікація** - це присвоєння якому-небудь об'єкту або суб'єкту унікального образу, імені або числа.

**2.** Встановлення дійсності (**автентифікація**) полягає в перевірці, чи є об'єкт (суб'єкт), що перевіряється, справді тим, за кого себе видає.

# Автентифікація в додатках

**Причини використання:**

- контролювати доступ до системи або додатку
- Щоб зв'язати деякі конфіденційні дані з людиною, наприклад, для шифрування
- Встановити довіру між декількома сторонами, щоб сформувати взаємодію між ними
- Гарантувати, що частина інформації є справжньою

У додатку можуть застосовуватися один або всі з цих аспектів .

# Необхідність автентифікації

- Якщо очікується наявність кілька користувачів або додатків, які будуть взаємодіяти з даними додатком, чи всі вони отримають доступ до одних і тих же даних, конфігурації та інформації, або у кожного буде свій власний набір даних, незалежно від того, наскільки вони малі?
- Чи працює додаток в повністю закритою і довіреної області, в якій немає людини, системи або додатків, які будуть отримувати до нього доступ з ненадійних частин, таких як Інтернет, інші мережі або невідомі додатки?
- Чи існує концепція конфіденційної інформації і її поділ або ізоляція в додатку?

Якщо відповідь на будь-який з цих питань «так», в застосунку необхідна автентифікація.

# Локальна автентифікація

Причини реалізації локальної автентифікації в додатку:

- Автономний додаток
- Немає зв'язку або поганий зв'язок
- Обмежені, невеликі або вбудовані додатки
- Обмежені ресурси програми

Якщо хтось вибирає, щоб додаток використовував власний механізм автентифікації, є кілька поширених підходів:

- Реалізувати власну схему автентифікації, засновану на стандартних методиках.
- покладатися на послуги локальної операційної системи, якщо такі є (Unix / Windows автентифікація)
- Використовувати доступні бібліотеки для забезпечення функціональності автентифікації

# Зовнішня автентифікація

Служби автентифікації на глобальному і корпоративному рівнях часто є частиною більш великих структур каталогів і служб. Вони включають:

- LDAP
- Active Directory
- NIS / NIS +
- Kerberos

Для роботи з цими службами і вже встановленими середовищами додатки повинні будуть використовувати різні бібліотеки і інтерфейси для роботи з цими системами та інфраструктурою.

# Злом облікового запису

► Більшість паролів підбираються за допомогою автоматизованих інструментів
(Brutus, THC Hydra, Web Brute, Bert, SqlPing, Wfuzz, Aircrack-NG).

► ***Метод грубої сили***

Самий складний і довгий шлях - методом перебору. У більшості випадків вимагає великих витрат з точки зору кількості необхідних припущень і часу, необхідного для успіху.

► ***Атаки за словником***

Середньостатистичний користувач використовує «кореневе слово» як мінімум як частину свого пароля. Робочий словниковий запас освіти людини становить **10 000** слів, зазвичай менше (**3000 - 4000** слів) і підбір пароля починається з перебору найбільш загальних слів. За різними оцінками дозволяє розшифрувати до **60-70%** призначених для користувача паролів на будь-якому загальнодоступному сайті

# Злом облікового запису

► Як що політика безпеки потрібно «складний» пароль, то як правило, більшість паролів будуть починатися з **великої приголосної**, за якою відразу ж слід **рядкова голосна** (зазвичай **а, і** чи **о**); якщо потрібно **число**, це буде **1** або **2**, розташовані **в кінці**; а якщо потрібно спеціальний символ, це буде **!, @, #, $** або **&,** і він буде розташований замість схожою літери.

**80 відсотків** пользователь мають пароль в аналогічному форматі.

► Чим **довше пароль**, навіть без спеціальних символів, тим **вище** у нього **ентропія**. Зіткнувшись з вибором: використовувати більше спеціальних символів або використовувати більш довгий пароль, **використовуйте довший пароль**. Він забезпечить більший захист і буде легше запам'ятовувати.

# Злом облікового запису

**Brutforce:**

► Підбір за принципом «грубої сили», що дозволяє розшифрувати **більше половини** паролів довжиною від одного **до шести символів** включно, в число яких входять по 26 латинських букв нижнього і верхнього регістрів, 10 цифр і 33 інших символів, в сумі **– 95 символів.**

► Паролі, що складаються тільки з **букв нижнього регістру**, довжиною **до 8** символів, а також паролі **з чисел** довжиною до **12 символів**. Метод «грубої сили» з такими параметрами дозволяє розшифрувати **значний відсоток** паролів

# Злом облікового запису

► Програми для **автоматичного підбору** пароля **блокують** багато облікових записів. Часто головною **першою ознакою** того, що програма для підбору пароля працює в мережі, є несподівана **блокування** майже всіх облікових записів одночасно.

► На комп'ютерах і в мережах **Windows** єдиний обліковий запис, що не може бути заблокований це запис «**Адміністратор**» ( (SID S-1-5-*домен*-500), тому її можна використовувати, щоб розблокувати облікові записи всіх інших. Оскільки її неможливо заблокувати, вона також є улюбленою метою для підбору паролів.

# Злом облікового запису

**Birthday Attack :**

► парадокс дня народження говорить, що при випадковому зібранні **23 людей існує 50%** шансів, що у двох людей буде той самий день народження.

► Якщо ми обираємо день народження як пароль, і зберігаємо паролі як хеші, то всі однакові паролі видають однаковий хеш, якщо ми використовуємо той самий алгоритм хешування.

► Атака дня народження шукає однакові хеши. Якщо він знаходяться два хеші однакового значення, зловмисник також знає, що пароль однаковий.

# Злом облікового запису

**Парольні атаки:**

- Замість того, щоб намагатися вгадати пароль для одного аккаунта тисячу чи десять тисяч разів, вони підбирають кілька разів пароль для тисячі або **десяти тисяч облікових записів**.
- Виконання атаки включає наступні кроки:

1) Зловмисник використовує інструменти **OSINT (open source intelligence tools)** наприклад **recon-ng**, щоб зібрати якомога більше **імен входу** в систему і зберегти їх у файлі.

2) Зловмисник знаходить доступний онлайн-портал входу в систему жертви (наприклад, Outlook для веб-доступу, Gmail, протокол віддаленого робочого стола RDP, Cisco VPN)

# Злом облікового запису

**Spraying Attack:**

- хакер перш за все шукає в Інтернеті людей, які працюють в організації, і бачить, чи зможуть вони допомогти розгадати стандартну конвенцію щодо імен.
- Потім вони переглядають Інтернет-ресурси, такі як Facebook, Twitter та LinkedIn, щоб знайти імена якомога більшої кількості співробітників організації та застосувати стандартну конвенцію щодо імен.
- Коли вони мають стандартну конвенцію іменування, вони застосовують це до кожного знайденого імені працівника.
- Потім вони використовують такий інструмент, як Kali Linux, і розпорошують (**Spraying)** пароль, що означає використання найпоширеніших паролів по одному за списком співробітників в надії, що один з них відповідає.

**Конвенція імен** – формат якому відповідають назви облікових записів
Наприклад **John.Daw** або **Daw.John**

# Злом облікового запису

***Ризик при повторному використанні пароля величезний***

► **Найбільший ризик** крадіжки паролів полягає в тому, що, хоча пароль може бути вкрадений **тільки з одного веб-сайту** або служби, він часто повторно використовується одним і тим же користувачем на **кількох веб-сайтах**

► У середнього користувача є **шість - сім паролів**, які вони використовують для кожного веб-сайту і служби. Якщо зламаний пароль до одного з них, можна почати перевірку того, чи працює ця ж інформація для входу на безліч інших більш важливих веб-сайтів, таких як Amazon, банки і т.п .. І будь-який хороший хакер автоматизує цей процес перевірки.

► Кілька веб-сайтів присвячені тому, щоб повідомити вам, чи зламані ваші паролі. Самим популярним і відомим є сайт Троя Ханта (**haveibeenpwned.com**). Він містить більше **8,5 мільярда** імен (2019 р.). для входу та паролів. Ви просто вводите свою **адресу електронної пошти**, і веб-сайт повідомить вам, скільки разів ваше ім'я для входу і пароль з'являються в його базі даних з усіх зібраних порушень пароля, про які він знає. Веб-сайт не тільки повідомляє вам, скільки разів ваше ім'я для входу в систему і пароль були скомпрометовані, але також надає вам веб-сайти, які були скомпрометовані, щоб отримати його, або дампи паролів, на яких він був виявлений.

# Основні вимоги до механізму аутентифікації

► **Контроль невдалих спроб доступу**.

Механізм ІА повинен надавати адміністратору можливість задавати максимальну кількість невдалих спроб доступу за заданий інтервал часу стосовно користувачеві або ролі.

► **Неможливість створення ролі без аутентифицирующей інформації** .

Якщо додаток використовує рольове управління доступом, він не повинен дозволяти створення ролі без пов'язаної з нею аутентифицирующей інформації (паролі, сертифікати, PKI ).

► **Аутентифікація груп / ролей**

Якщо в додатку використовується аутентифікація на рівні груп або ролей, то перш ніж користувач буде авторизований як член групи або ролі, він повинен пройти індивідуальну процедуру ІА.

# Основні вимоги до механізму аутентифікації

- **ІА інших серверів**

В процесі обробки запитів користувачів додаток може взаємодіяти з іншими серверами, наприклад, серверами баз даних. Додаток не повинен обмежувати можливість користувачів взаємодіяти з механізмом ІА таких серверів. Для захисту таких серверів, механізм ІА повинен використовувати тільки акредитований сервер аутентифікації або SSO ( single signed authentication)

- **Межпроцессной аутентифікація**

Прикладні процеси, що діють від імені користувачів, повинні бути автентифіковано сервером, до якого вони питиются отримати доступ. Рівень строгості аутентифікації повинен бути не нижче, ніж той, який використовується при аутентіфкації користувачів.

# Основні вимоги до механізму аутентифікації

- **База безпеки.**

Паролі та інша інформація, яка використовується в процесі аутентифікації, повинні бути надійно зашифровані перед збереженням. База збереженої інформації повинна бути захищена від видалення або модифікації.

- **Аутентифікація осіб з адміністративними повноваженнями.**

Вимоги до аутентифікації осіб з адміністративними повноваженнями повинні бути більш суворими (наприклад, вимоги до довжини і складності пароля);

- **Аутентифікація кожної сесії користувача .**

Кожен раз, при ініціалізації нової сесії користувач повинен вводити пароль. Додаток не повинен зберігати пароль в файлах cookies , або програмах ( скриптах ) на стороні клієнта або сервера, що дозволяє користувачеві не вводити пароль при ініціалізації нової сесії.

# Основні вимоги до механізму аутентифікації

## Критичні уразливості сервісу аутентифікації

- Паролі передаються в незашифрованому вигляді
- Для зберігання паролів використовується ненадійний метод шифрування
- Паролі «за замовчуванням» після розгортання програми не змінені
- При введенні пароля він відображається у вікні введення
- Ауттентіфікаціонние дані зберігаються в додатку
- Облікові записи користувачів не блокуються після заданої кількості невдалих спроб введення пароля
- Структури пам'яті, що містять аутентифікаційні дані не очищаються після завершення сесії користувача.
- Досить серйозна уразливість створюється також при використанні слабких паролів.
- Для аутентифікації допускаються недійсні або прострочені сертифікати PKI

# Аутентифікація за допомогою паролів

**Механізм управління паролями** повинен дозволяти адміністратору:

- призначати користувачам паролі і встановлювати **термін** їх життя,
- встановлювати режим необхідності **зміни пароля** користувачем при першому доступі до додатка
- встановлювати режим перевірки **історії паролів** , який дозволить користувачам використовувати попередні паролі;
- встановлювати **обмеження** на паролі: довжина пароля, наявність символів у верхньому і нижньому регістрі, неалфавітних символів;
- встановлювати **обмеження на безлічі паролів** , які користувач використовувати не може: слова зі словника, набір символів, що повторюються, імена облікових записів і т.п ;
- встановлювати **допустиму кількість невдалих спроб** аутентифікації; Якщо користувач перевищив допустиму кількість спроб, його обліковий запис повинен **блокуватися** на певний час.

# Оцінка якості паролю по ентропії

Вимога складності пароля - це спроба запобігти створенню пароля з надзвичайно популярних і легко вгадується або легко зламувати паролі.

- Показник випадковості пароля називається **ентропія**. Ентропія пароля є мірою його істинної випадковості.

$$H = L\frac{lnN}{ln2} = ln_2N^L$$

де **N** - розмір алфавіту, **L -** довжина паролю

- По-справжньому випадковий пароль, який може бути обраний програмою-менеджером паролів, вважається у яких **високу ентропію**. Паролі з високою ентропією складніше створити, запам'ятати і використовувати.
- Вважається, що паролі, які вибирає більшість людей, мають **низьку ентропію**. Паролі з низькою ентропією **легше** вгадати і зламати.

# Оцінка якості паролю по ентропії

- Методика NIST для оцінки ентропії пароля, **придуманого людиною** і що не включає символи з неанглійських алфавітів. Алгоритм заснований на тому припущенні, що люди вибирають паролі з тієї ж ентропією, що і звичайний англійський текст:

- ентропія **першого** символу приймається рівною **4** бітам;
- ентропія **наступних семи символів** приймається рівною по **2** біти на кожен символ;
- ентропія символів від **9-го до 20-го** приймається рівною по **1,5** біта на кожен символ;
- ентропія пароля приймається рівною **сумі ентропій** символів;
- якщо в паролі використовуються **великі літери і неалфавітні символи**, то до суми додається ще **6** біт.

Для пароля, з **8 символів**, що не містить заголовних букв і не містить неалфавітних символів, складе близько **4 + 7 * 2 = 18 біт**.

# Ентропія згенерованого коду

- Для оцінки ентропії **згенерованого пароля** можна скористатися таблицею

| Набор символів | Кількість символів, N | Ентропія на один символ, біт |
|---|---|---|
| Арабськи цифри (0-9) | 10 | 3,3219 |
| Цифри шестнадцатеричної системи числення (0-9, A-F) | 16 | 4,0000 |
| Малі літери латинського алфавіту (a-z) | 26 | 4,7004 |
| Арабські цифри і малі літери латинського алфавіту (a-z, 0-9) | 36 | 5,1699 |
| Рядкові і заголовні букви латинського алфавіту (a-z, A-Z) | 52 | 5,7004 |
| Арабські цифри, рядкові і заголовні букви латинського алфавіту (a-z, A-Z, 0-9) | 62 | 5,9542 |
| Всі друковані символи ASCII | 95 | 6,5699 |

**H >= 64 b**

# Оцінка якості паролю по Імовірності підбору пароля

$$P = \frac{T * N}{A^m}$$

Приклад:

T= 60 днів або 5184000 сек.

N = 100 000 в сек.

A = 93 (Латинський алфавіт н/р + в/р = 52 + цифри = 10 +спец. сімволи = 31)

M= 8

P = (2,592 000 * 100 000 000) / $93^8$ = 0.93 *$10^{-4}$

**P = $10^{-6}$** звичайні системи

**P = $10^{-15}$** системи в яких обробляється конфіденційна інформація

# Аутентифікація за допомогою паролів

Типова політика щодо паролів:

- Термін дії паролів закінчується через **60 днів**.
- Паролі мають містити щонайменше **15 символів**.
- Паролі повинні містити принаймні **одну велику літеру**.
- Паролі повинні містити принаймні **одну малу літеру**.
- Паролі повинні містити принаймні **одну цифру**.
- Паролі повинні містити принаймні **один спеціальний символ**.

# Автентифікація за допомогою паролів

Нова політика паролів NIST Digital Identity Guidelines ( pages.nist.gov/800-63-3/sp800-63b.html ) 2015 р.

► До сих пір, жоден з найбільших світових органів щодо забезпечення комп'ютерної безпеки (PCI-DSS, HIPAA, NERC, CIS та ін.) не рекомендує нові правила. Microsoft, яка активно просувала старі рекомендації по політиці паролів, припинила рекомендувати якусь конкретну політику паролів.

Однак NIST опублікував список поширених питань (FAQ) у 2020 році, щоб уточнити свої рекомендації. З цими роз’ясненнями ми очікуємо, що більше федеральних агенцій виконуватимуть ці рекомендації.

# SP 800-63B Рекомендації щодо цифрової ідентифікації

- Паролі ПОВИННІ бути не менше **8 символів**, якщо вони **вибираються абонентом.**

- Паролі, **вибрані випадковим чином** CSP або верифікатором, повинні бути довжиною **не менше 6** символів і можуть бути повністю цифровими.

- Якщо CSP або верифікатор забороняє вибраний пароль на підставі його появи в чорному списку, абонент ПОВИНЕН вибрати інший пароль.

- Ніяких інших вимог до складності паролів, **що запам'ятовуються**, НЕ ПОВИННО пред'являтися

# SP 800-63B Рекомендації щодо цифрової ідентифікації

- **Термін дії паролів не повинен закінчуватися**. Користувачі не повинні вимагати змінювати свої паролі регулярно, наприклад, кожні 30 днів. Користувачі часто змінювали один символ під час примусу щоб змінити свій пароль. Наприклад, вони змінюють Password1 на Password2. Хоча це відповідає вимогі змінити пароль, це не додає безпеки. Зловмисники використовують ті ж методи під час вгадування паролів.
- **Користувачі не повинні використовувати спеціальні символи**. Вимагання від користувачів включати спеціальні символи часто викликало виклик пам’яті користувачів, і вони записували ці паролі. Крім того, NIST проаналізував зламані бази даних паролів і виявив, що спеціальні символи в паролях не дають бажаних переваг.
- **Користувачі повинні мати можливість копіювати та вставляти паролі**. Менеджери паролів дозволяють користувачам створювати та зберігати складні паролі. Користувачі вводять один пароль у менеджері паролів для доступу до збережених паролів. Потім вони можуть копіювати паролі з менеджера паролів і вставляти паролі в текстове поле пароля. Коли копіювання та вставлення обмежено, користувачі повинні повторно ввести пароль і зазвичай за замовчуванням використовують простіші паролі.

# SP 800-63B Рекомендації щодо цифрової ідентифікації

► **Користувачі повинні мати можливість використовувати всі символи**. Механізми зберігання паролів зазвичай мають відхилені пробіли та деякі спеціальні символи. Дозволяючи пробіли, користувачі можуть створюватидовші паролі, які легше запам’ятати. Системи іноді відхиляють спеціальні символи, щоб запобігти атакам (наприклад, атака ін'єкції SQL), але належне хешування пароля маскує ці символи.

► **Довжина пароля повинна бути не менше восьми символів і не більше 64 символів**. Більша довжина дозволяє користувачам створювати значущі для них парольні фрази.

► **Системи паролів повинні відображати паролі**. Перш ніж прийняти пароль, пароль системи повинні звіряти їх зі списком часто використовуваних паролів, наприклад 12345 пароль.

► Відмовтеся від будь-яких паролів, які знаходяться в **чорному списку**, який підтримує CSP. Чорний список повинен включати наступне:

- Паролі, отримані з попередніх випадків  порушень.
- Словникові слова.
- Повторювані або послідовні символи (наприклад, аааааа, 1234abcd).
- Контекстні слова, такі як ім'я користувача та похідні від них.
- Будь-які тривіальні зміни пароля з чорного списку

# Автентифікація за допомогою паролів

**PCI DSS Password Requirements**

PCI DSS (версія 3.2.1) має такі вимоги, які відрізняються від NIST SP 800-63B:

- Термін дії паролів закінчується щонайменше кожні **90 днів**.
- Паролі мають містити не менше **семи символів**.

Якщо організаціям потрібно відповідати певному стандарту, такому як PCI DSS, вони повинні дотримуватися принаймні мінімальних вимог цього стандарту.

.

# Зберігання паролів

**H = H ( pwd)**

- Основний спосіб зберігання пароля - хешування
- Існує безліч стандартів криптографічного хешування минулому і в даний час, включаючи MD5, Windows LANManager (**LM**), Windows NT (**NTLM**), функцію отримання ключа на основі пароля (**PBKDF**), Secure Hash Algorithm (**SHA-1, SHA-2, SHA-3**) і **Bcrypt**.
- Сьогодні хеші **NTLM**(без солі) і **PBKDF2** (з сіллю) за замовчуванням використовуються для хешування паролів на комп'ютерах з **Windows**
- **SHA-2** або **Bcrypt** на комп'ютерах в стилі **Linux / BSD / Unix**

# Атаки на хеші паролів

- Хеші паролів майже так само цінні, як і паролі у вигляді відкритого тексту.
- Хакери можуть часто використовувати їх в атаках з передачею хеша (**Pass-the-hash PtH**). Вони найбільш популярні в системах MS Windows (хеші NTLM не містять солі).
- Програми для отримання хешу (Pwdump, Mimikatz, Metasploit і Empire PowerShell Toolkit). Всі ці інструменти вимагають як мінімум доступу локального адміністратора на комп'ютері з Windows;
- крадіжкою хешу за допомогою email. Якщо посилання в листі веде на сайт, що вимагає автентифікації, то хеш відправляється автоматично (`www.securify.nl/blog/SFY20180501/living-off-the-land_-stealing-netntlmhashes.html`)
- Зловмисники, якщо вони отримають доступ тільки до хешів паролів, конвертують якомога більше з них в їх еквіваленти у вигляді відкритого тексту. Наприклад, за допомогою «райдужних» таблиць

# Атаки на хеші паролів

► Хакери могут часто використовувати їх в атаках з передачею хешу (**Pass-the-hash PtH**).

У Windows типовий хеш виглядає приблизно так:
**admin2: 1000: aad3b435b51404eeaad3b435b51404ee: 7178d3046e7ccfac0469f95588b6bdf7**

Рядок вище складається з чотирьох секцій, розділених двокрапкою.
- Перша частина - **ім'я користувача**,
- друга - **Id**.
- Третя частина являє собою **LM хеш**, що припинив використовуватися, починаючи з Windows Vista / Server 2008.
- Четверта частина являє собою **NTLM хеш**.

Паролі не передаються по мережі у відкритому вигляді, **а хешуються відразу ж після введення під час автентифікації**. Можна під час автентифікації скористатися хешем, замість пароля в чистому вигляді. Особливо небезпечно, коли відомо ім'я користувача з адміністративними правами і хеш.

# Атаки на хеші паролів

► **Rainbow table**

Зловмисник генерує великий словник можливих паролів. Для кожного пароля зловмисник генерує хеш-значення, пов’язані з кожним можливим значенням солі. Результатом є величезна таблиця хеш-значень, відома як райдужна таблиця. **Райдужні таблиці** - це форма попередньо обчислених **хеш-таблиць**. Райдужні таблиці використовують переваги концепції, відомої як **хеш-ланцюжок**. Райдужні таблиці мають свої обмеження. Розмір райдужної таблиці залежить від діапазону можливих значень паролів. Для поганих алгоритмів хешування паролів та коротких простих паролів таблиця веселки може бути досить маленькою, але для надійного хешування та складних паролів таблиця стає незрівнянно великою.

► Наприклад, використовуючи 1,4 ГБ даних, він міг зламати 99,9% усіх алфавітно-цифрових хешів паролів Windows за 13,8 секунди. Цьому підходу протистоять використання достатньо великого значення солі та достатньо великої довжини хеша

► Можна захиститись від райдужних таблиць (та інших атак злому паролів), застосувавши **сіль**.

# «Солоне» хешування паролів

► Якщо зловмисник отримує доступ до результату хешування, у нього не повинно бути можливості знайти вихідний введення **швидше, ніж атака за словником**.

► Зловмисник не повинен дізнаватися, що у різних користувачів один і той же пароль або що один і той же пароль був у одного користувача в різний час. Функція хешування повинна видавати **різні результати для однакових вхідних даних**. Це досягається включенням **випадковості** в хеш-функцію через **сіль**.

► Повинно бути складно виконати словникову атаку на багато паролі з невеликими витратами. Це досягається за рахунок використання **спеціальної функції**, розробленої спеціально для хешування паролів.

► Різні паролі не повинні давати один і той же хеш з досить високою ймовірністю (колізії).

# «Соління» паролів

**H = H ( pwd || salt)**

**H = H (pwd)|| H(salt)**

**H = H (pwd|| H(salt))**

- Сіль повинна бути згенеровано за допомогою стійкого генератора псевдовипадкових чисел
- Сіль повинна бути **унікальна** - своя для кожного користувача і пароля.
- Сіль повинна бути **довгою** (80 - 256 біт або 10 - 32 символів) Слід використовувати сіль, щонайменше, **тієї ж довжини, що і вихід хеш-функції**. За рекомендацією NIST >= 32 символа
- Сіль повинна бути **збережена** в таблиці облікових записів користувачів разом з хеш-кодом
- У веб-додатках завжди обчислюйте хеш-код **на сервері**

# Розтягування пароля (password stretching )

► **Розтягування пароля (password Stretching** )

Перевіреним методом є розтягування паролів до їх збереження в базі даних. Основна мета розтягування пароля - **зробити розшифровку пароля дорожчою** - збільшення обчислювального часу в системі зловмисника.

► При розтягуванні **сила пароля** вимірюється його бітами міцності ключа.

► Методи подовження кількості бітів пароля зводиться до **хеш-функції.** Зазвичай хеш-функції **циклічно повторюються** тисячі разів, імітуючи випадковість і додаючи все більше і більше бітів до складності пароля, переданого в базу даних.

► Популярні інструменти для розтягування паролів є **PBKDF2** та **Bcrypt**. На сьогоднішній день 128-розрядні ключі є загальним еталоном ефективного шифрування паролів.

# «Соління» паролів

► Зробити хеш-функцію дуже повільної, щоб навіть за допомогою спеціального обладнання, атаки повним перебором були занадто повільними

$$H_0 = H(pwd, H(salt))$$

$$H_n = H(H_{n-1}, H(salt))$$

Хеш $H_n$ зберігається разом із значенням **salt** . У процесі верифікації додаток повторять процедуру обчислення хешу і звіряє отримане значення $H_n$ з збереженим.

► Для настільного програмного забезпечення або додатків для смартфонів найкращий спосіб вибрати параметр **n** - це запустити короткий тест на пристрої, щоб знайти величину, при якій хеш-код обчислюється приблизно **півсекунди** . За рекомендацією NIST n >= 10 000 - кількість ітерацій.

# «Соління і розтягування» паролів

► **Розтягування паролів-** це сукупність прийомів, які передбачає додавання **ітеративних обчислень**, що збільшує складність злому паролю.

► **Bcrypt** - приклад технології розтягування ключів. Він заснований на шифрі Blowfish, використовує **засолювання** та включає адаптивну функцію для збільшення ітерацій з часом.

► Функція виведення ключів на основі пароля (**PBKDF2**) - використовує операцію хешування, функцію шифрування або операцію HMAC (симетричний ключ використовується в процесі хешування) на вхідному паролі, який поєднується з **сіллю**. Потім цей процес повторюється тисячі разів.

# «Соління і розтягування» паролів

Функції хешування паролів (Bcrypt ...) приймають такі вхідні дані:

- **пароль** - за згодою закодовані в UTF-8.
- **сіль** являє собою масив випадкових байтів (зазвичай 16 або 32 байта).
- **Вартість** – параметр що настроюється. Визначає, скільки часу або ресурсів комп'ютера потрібно для створення вихідних даних. Для різних алгоритмів хешування паролів це по-різному: одні приймають одне число (к-ть ітерацій $2^n$), інші - більше параметрів.
- сервер збереже хеш пароля разом з сіллю і вартістю. Сіль і ціна не шифруються і не хешіруются - вони зберігаються як є. Щоб перевірити пароль, сервер витягує хеш, сіль, вартість, а потім запускає функцію хешування пароля ..
- Практичне правило хешування паролів: використовуйте найшвидшу реалізацію самого повільного алгоритму

# Додавання «перцю» до паролю

- Перець (pepper ) - це секретна значення, що додається до пароля **перед хешуванням**. На відміну від солі, воно **не зберігається в базі даних разом із хешами**.
- Зазвичай перець закодований у вихідний код веб-сайту (**site-wide static value**) і вважається секретним.

$$H_0 = H(pwd||papper, H(salt))$$

$$H_n = H(H_{n-1}, H(salt))$$

- Найбільшою перевагою перцю є той факт, що **він не зберігається в базі даних**. Крадіжка бази даних паролів не призводить до того, що вся таблиця паролів буде зламана.
- Однак, оскільки перець є жорстко закодованим, це, як правило, **однаковий набір біт даних** для кожного пароля, і його не можна змінити.
- За рекомендацією NIST це має бути обов'язково.

# Популярні програми хешування паролів

► **PBKDF2** . Популярний головним чином тому, що існує вже давно, він затребуваний різними бюрократичними стандартами і широко доступний для багатьох мов програмування. Однак **це найслабша** з функцій хешування паролів

► Twitter і GitHub використовують **bcrypt** для хешування паролів. Використовує **16-байтові солі**

► **Scrypt** - це функція хешування паролів, яка, крім обчислень, які споживають ресурси процесора, також використовує певний обсяг пам'яті для виконання обчислень.

► **Argon2** - це функція хешування паролів і отримання ключів, яка перемогла **в конкурсі з хешування паролів в 2015 році**. Його можна окремо налаштувати в залежності від того, скільки пам'яті він споживає і скільки часу займає обчислення.

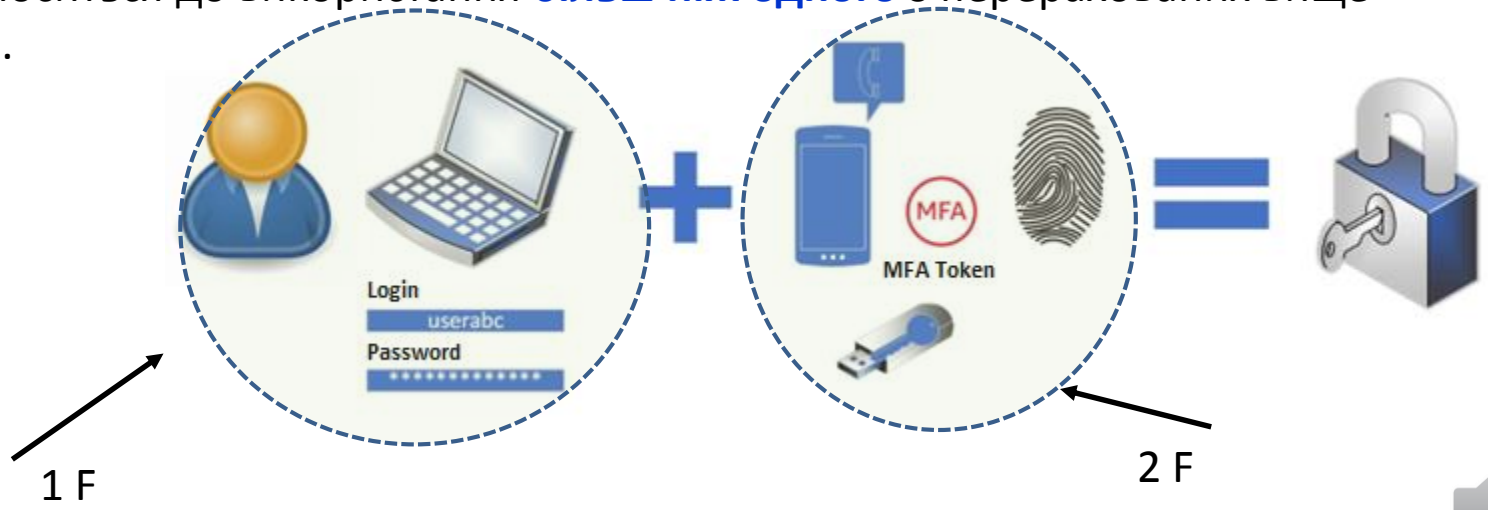
# Багатофакторна автентифікація (Multifactor authentication -MFA)

# Багатофакторна автентифікація (MFA)

Фактори автентифікації :

1. **Type 1. Щось, що ти знаєш** Включає інформацію, яку ви можете згадати з пам'яті. Приклади включають **пароль, код, PIN-код**, комбінацію або секретну фразу.
2. **Type 2. Щось, що у вас є** Вимагає володіння та використання фізичного предмета. Приклади включають **смарт-карту, пристрій** або ключ.
3. **Type 3. Щось є частиною тебе**, часто називають **біометрією**. Приклади включають відбитки пальців, сканування сітківки або розпізнавання голосу. Раніше це було відомо як фактор типу 3.

MFA відноситься до використання **більш ніж одного** з перерахованих вище факторів.



# Багатофакторна автентифікація

**Додаткові елементи**, які можна включати та оцінювати при визначенні ідентичності, автентифікації та навіть авторизації суб’єкта.

► **Десь ви знаходитесь** - підтвердження на основі місцезнаходження. Приклади включають фізичне місцезнаходження або логічну адресу, наприклад ім'я домену, IP-адресу або MAC-адресу.

► **Щось ти можеш зробити** - передбачає певні навички чи дії, які ви можете виконати. Може включати такі дії, які є біометрично виміряними та напівдовільними, такі як ваш **ритм друку**, поведінка при ходьбі (тобто **хода**), **почерк**, вирішення CAPTCHA або **поведінка миші**.

► **Щось, що ви демонструєте**, фокусується на якомусь факті, який можна виявити щодо предмета чи їхнього зв’язку. Сюди може входити таке: зв’язок є відкритим текстом чи зашифрований, відома система чи невідома, встановлені останні виправлення чи ні, яка ОС використовується.

► **Хтось, кого ви знаєте** - це автентифікація за ланцюжком довіри. Це коли ви отримуєте запрошення приєднатися до групи або отримати доступ до послуги від когось, хто вже є учасником

Додаткові атрибути можна використовувати для оцінки ризику ідентифікації, але вони не вважаються факторами автентифікації

# Вибір кількості факторів

| Категорії загрози | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1. Незручності або пошкодження репутації | None | Low | Moderate | High |
| 2. Фінансові втрати або відповідальність | None | Low | Moderate | High |
| 3. Шкода для проектів або суспільних інтересів | None | Low | Moderate | High |
| 4. Несанкціоноване розкриття конфіденційної інформації | None | Low | Moderate | High |
| 5. Особиста безпека (персональна інформація) | None | Low | Moderate | High |
| 6. Цивільні чи кримінальні правопорушення | None | Low | Moderate | High |

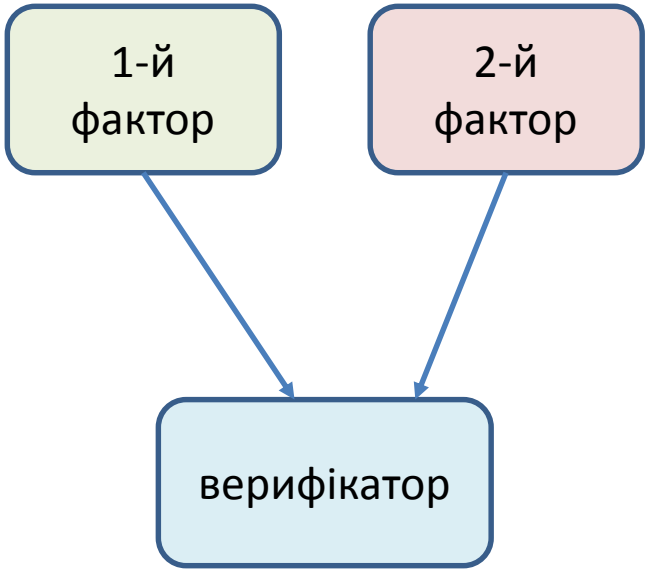
Або (1,2) - Low

Або (3,4,5,6) - Low

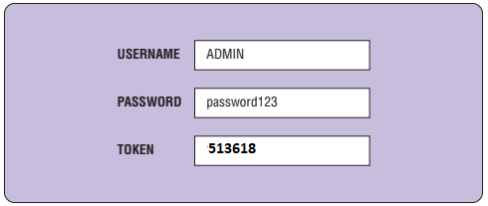
Або особиста безпека (5) - помірна

| AAL1 | AAL2 | AAL3 |
|---|---|---|
| **однофакторна** автентифікація | **двофакторна** автентифікація | **Підтвердження володіння ключем**. Автентифікація ПОВИННА використовувати **апаратний автентифікатор**, який забезпечує стійкість до імітації. **Двофакторна** автентифікація |

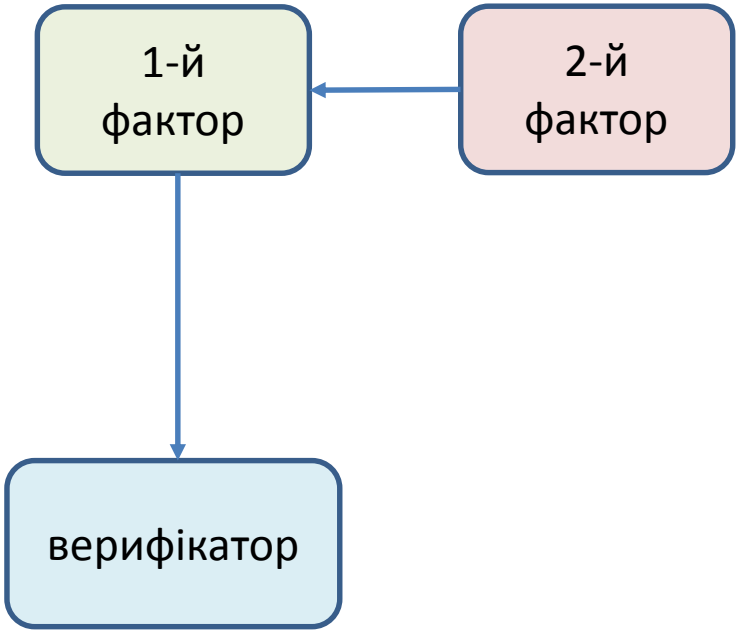
# MFA перша форма реалізації




Веріфікатору представлено кілька факторів;
Наприклад, поєднання запам'ятованого секрету (те, що ви знаєте) із пристроєм поза зоною доступу - смартфон (те, що у вас є).
Обидва атрибути передаються верифікатору для перевірки автентичності заявника.



# MFA друга форма реалізації



Деякі фактори можуть бути використані для захисту секрету, який буде представлений веріфікатору.

- Наприклад, апаратний засіб (автентифікатор), що містить криптографічний ключ (секрет автентифікатора), доступ до якого захищений відбитком пальця.
- - При використанні біометричних даних криптографічний ключ видає результат, який використовується для автентифікації заявника

# Токени

**OTP - One Time Password** - використання різних паролів при кожному новому запиті на надання доступу.

**Область застосування одноразових паролів**

- Автентифікація при віддаленому доступі через незахищені канали зв'язку (при можливому перехопленні автентификаційній інформації)
- Автентифікація з тимчасових робочих місць, де немає можливості встановлювати ПЗ і середовище не є контрольованою
- Як другий фактор в багатофакторной автентифікації.
- Автентифікація з пристроїв, які не мають можливості підключення смарт-карт / USB-ключів (КПК, смартфони та ін.).

# Токени

► Пристрої OTP зазвичай мають **глобальні унікальні ідентифікатори**, які жорстко запрограмовані в пристрої і також зберігаються в базі даних автентифікації серверної частини.

► Пристрої OTP мають секретний ключ. Такий же ключ зберігається на сервері

► **Початкове значення** генерується випадковим чином пристроєм і / або базою даних і передається серверу. За рекомендацією NIST **nonce** повинно бути не менш **32 символи**

► За допомогою секретного ключа, **nonce (або годинника реального часу)** і іншої наданої інформації (можливо, **унікальний ідентифікатор**) генеруються всі майбутні коди

► Вихідні дані токена можуть бути усічені до **6 десяткових цифр** (приблизно **20 біт** ентропії). Однак, як що вихідне значення менш **64** біт ентропії, сервер повинен обмежувати кількість невдалих спроб автентифікації. (NIST)

# Типи токенів

Токени

Контактні

Безконтактні

Повинні бути фізично підключені до комп’ютера. Токени цієї категорії автоматично передають інформацію про автентифікацію на клієнтський комп’ютер після встановлення фізичного з’єднання. Однак, щоб використовувати токен, необхідно встановити відповідний пристрій введення. Найпоширенішими типами фізичних маркерів є **смарт-карти** та **USB-токени**, які потребують зчитувача смарт-карт і порту USB відповідно.

Не мають ні фізичного, ні логічного з’єднання з клієнтським комп’ютером. Не вимагають спеціального пристрою введення, а використовують вбудований екран для відображення згенерованих даних автентифікації, які користувач вводить вручну за допомогою клавіатури. Є **найпоширенішим типом** токенів, який використовується (зазвичай у поєднанні з паролем) у двофакторній автентифікації для ідентифікації в Інтернеті.

# Контактні токени (фізичне з'єднання)

USB-C

USB-A-Nano

USB-A

Smartcard

# Безпровідні токени (логічне з'єднання)

**Bluetooth Google Titan**

**Bluetooth**

- До 10 м.
- Передача Bluetooth ідентифікаційних даних має найнижчий рівень безпеки автентифікації.
- Двонаправлене з'єднання для обміну транзакційними даними підвищує безпеку автентифікації.
- Токени Bluetooth часто поєднуються з токенами USB або смарт-картами, таким чином вони працюють як у підключеному, так і у відключеному стані.

**NFC token (смартфон)**

**Токени NFC**

- Автентифікація NFC працює, коли ближче ніж 1 фут (0,3 метра

**RFID token**

- Низький рівень захисту (перехоплення)
- LF (125—134 кГц)
- HF (13,56 МГц)

# Типи токенів

Токени

Синхронні

Асинхронні

Через певний проміжок часу (наприклад, кожні 30 секунд) токен і служба автентифікації переходять до наступного одноразового пароля . Пристрій відображає пароль користувачеві, який потім вводить це значення та ідентифікатор користувача. Служба автентифікації розшифровує облікові дані та порівнює OTP із очікуваним значенням. Якщо обидва збігаються, користувач проходить автентифікацію і отримує доступ до системи

Для автентифікації користувача використовується схема запит/відповідь. Сервер автентифікації надсилає користувачеві випадкове значення (**nonce)**. Користувач вводить це значення в токен, який шифрує його і повертає значення, яке користувач використовує як пароль. Користувач надсилає це значення разом з іменем користувача на сервер автентифікації. Сервер розшифровує значення і порівнює з тім, що надіслане раніше.

# Синхронні (RSA TOTP SecureID)

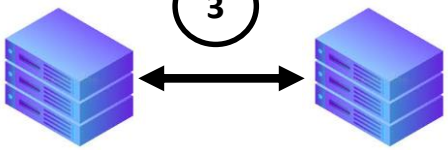
User Id: Alex
PIN: kmv14
OTP: 6234159

User Id: Alex
PIN: kmv14
OTP: 6234159

2

RSA authentication agent

3



RSA authentication manager

5

VPN, WEB, SSL-VPN…….

4

1

Час

Алгоритм

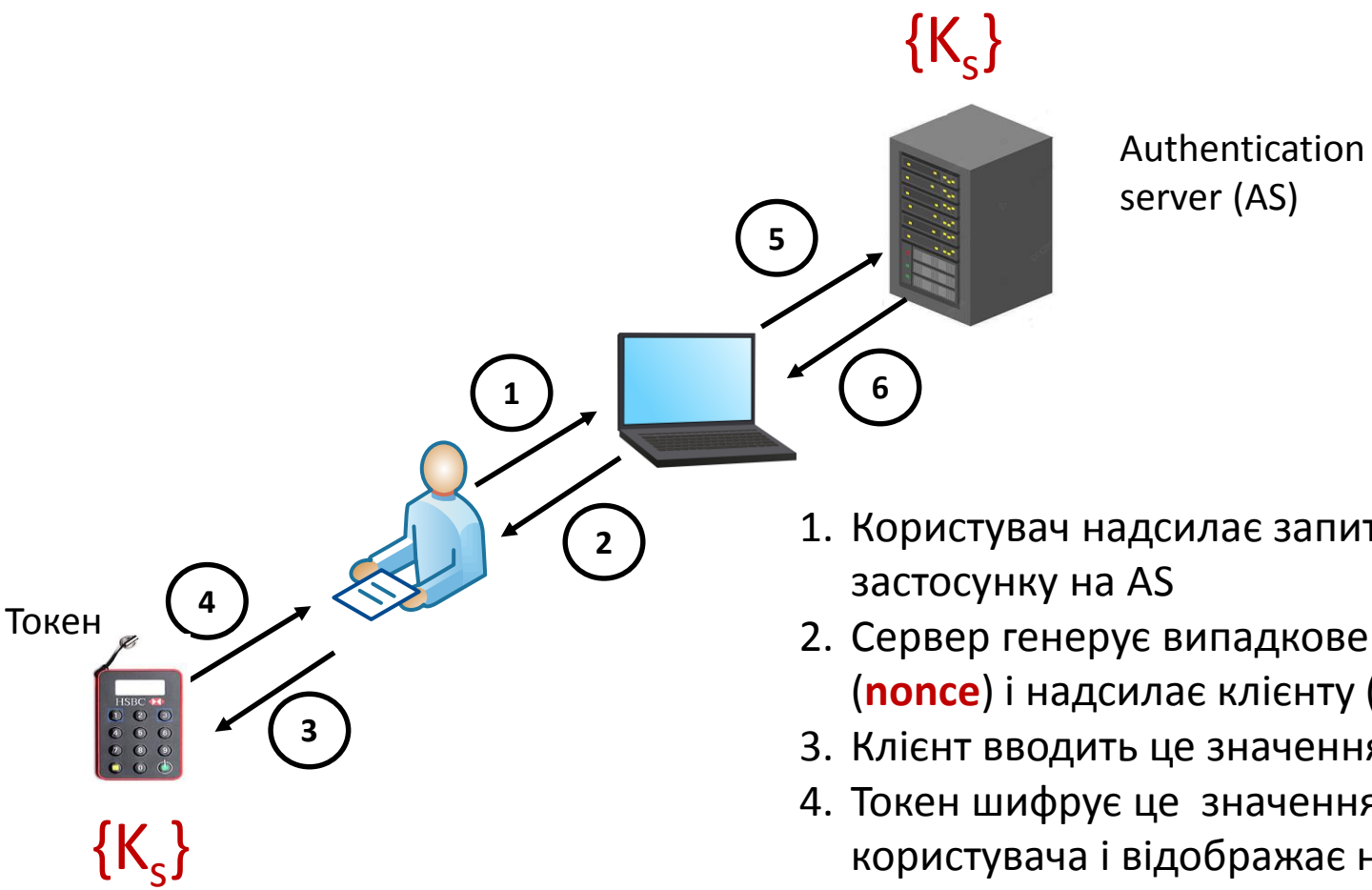
Seed 128 bit

Той же час

Той же Seed

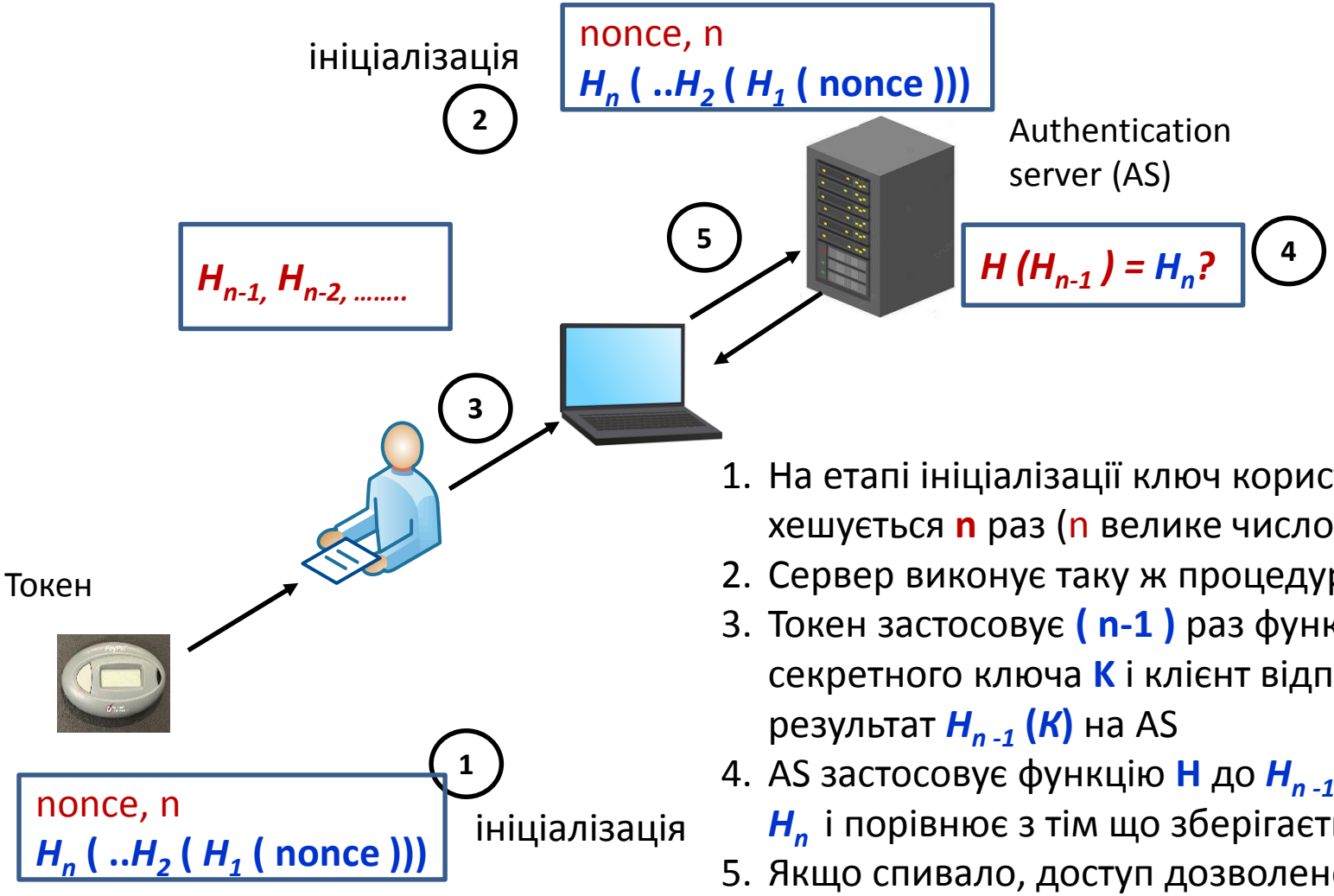
Той же алгоритм

# Асинхронний OTP (запит / відповідь)



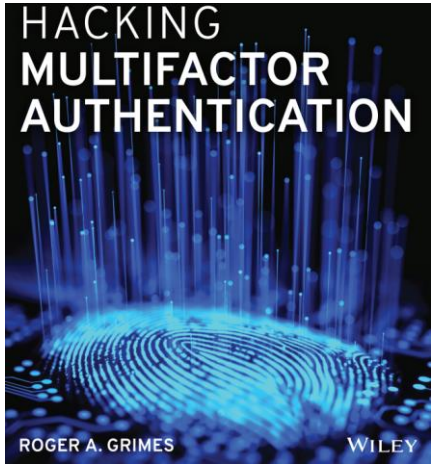

1. Користувач надсилає запит на доступ к застосунку на AS
2. Сервер генерує випадкове значення (**nonce**) і надсилає клієнту (на монітор)
3. Клієнт вводить це значення в токен
4. Токен шифрує це значення ключем користувача і відображає на екрані
5. Клієнт вводить це значення в комп'ютері і відправляє серверу автентифікації
6. AS шифрує **nonce** ключем користувача, порівнює з отриманим і дозволяє доступ

# Асинхронний OTP (тільки відповідь)




1. На етапі ініціалізації ключ користувача хешується **n** раз (n велике число)
2. Сервер виконує таку ж процедуру
3. Токен застосовує **( n-1 )** раз функцію **H** до секретного ключа **K** і клієнт відправляє результат $H_{n-1}$ (*K*) на AS
4. AS застосовує функцію **H** до $H_{n-1}$, отримує $H_n$ і порівнює з тім що зберігається
5. Якщо спивало, доступ дозволено. AS зберігає $H_{n-1}$
6. Зменшує на одиницю лічильник **(n)**

# Атаки на OTP




► **Технічні атаки**

Технічна атака - це пошук вада в цифровій технології, що лежить в основі безпеки рішення.

- злом шифрування, що захищає секрети,
- пошук передбачуваних закономірностей у тому, що має бути справді випадковою інформацією,
- Виявлення випадкових копій відкритого тексту того, що має бути зашифрованою інформацією,
- компрометація залучених кінцевих точок та інших учасників;
- підслуховування каналів зв'язку та інші подібні атаки.

► **Людський фактор**

- від **70 до 90 відсотків** усіх успішних зловмисних зломів посідає соціальна інженерія.
- Причиною **від 20 до 40** відсотків усіх зловмисних зламів даних є непропатчене програмне забезпечення
- Всі інші причини злому - неправильна конфігурація, підслуховування, спотворення даних - у сукупності становлять лише від **1 до 10 відсотків** .

-► **Фізичні**

- кражі
- обманом отримати доступ (телефон), де вони генеруються

# Біометрична автентифікація

- Біометричні характеристики - це унікальні особисті якості, які можуть бути використані для перевірки особистості людини, яка фізично присутня в точці перевірки. До них відносяться риси обличчя, відбитки пальців, малюнок райдужної оболонки ока, відбитки голосу та інші характеристики

**Критерії для біометричних параметрів.**

- **Загальність:** Даний ознака повинен бути присутнім у всіх людей без винятку.
- **Унікальність** : Біометрія заперечує існування двох людей з однаковими фізичними і поведінковими параметрами.
- **Сталість:** для коректної автентифікації необхідно сталість у часі.
- **Вимірність:** фахівці повинні мати можливість виміряти ознака будь-яким пристроєм для подальшого занесення в базу даних.
- **Прийнятність:** суспільство не повинно бути проти збору і вимірювання біометричного параметра.

- **BioAPI - 20 параметрів людини**

# Імовірнісний характер автентифікації

| | | Автентифікація успішна? | |
|---|---|---|---|
| | | Так | Ні |
| Легальний користувач? | Так | TP | FN |
| | Ні | FP | TN |

True positive

False negative

False positive

True negative

$$\text{False Positive (FAR)} = \frac{\text{False Positives}}{\text{False Positives + True Negatives}}$$

**Специфічність= 1 - FAR**

$$\text{False Rejection Rate (FRR)} = \frac{\text{False Negatives}}{\text{True Positives + False Negatives}}$$

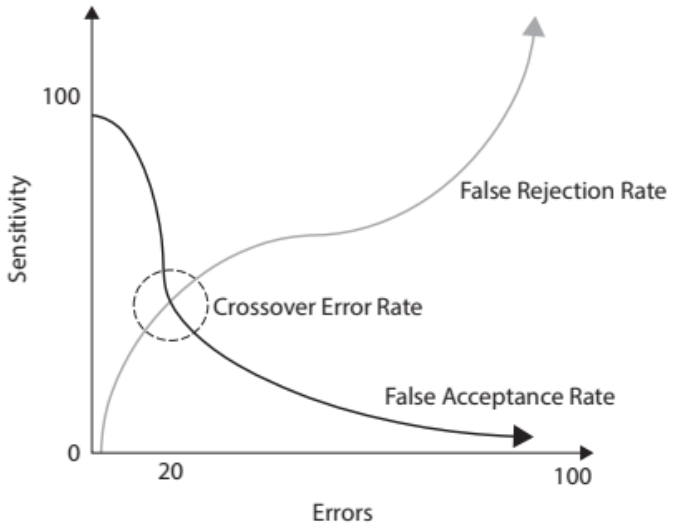
**Чутливість = 1 -FRR**

# біометрична аутентифікація

## Характеристики автентифікації

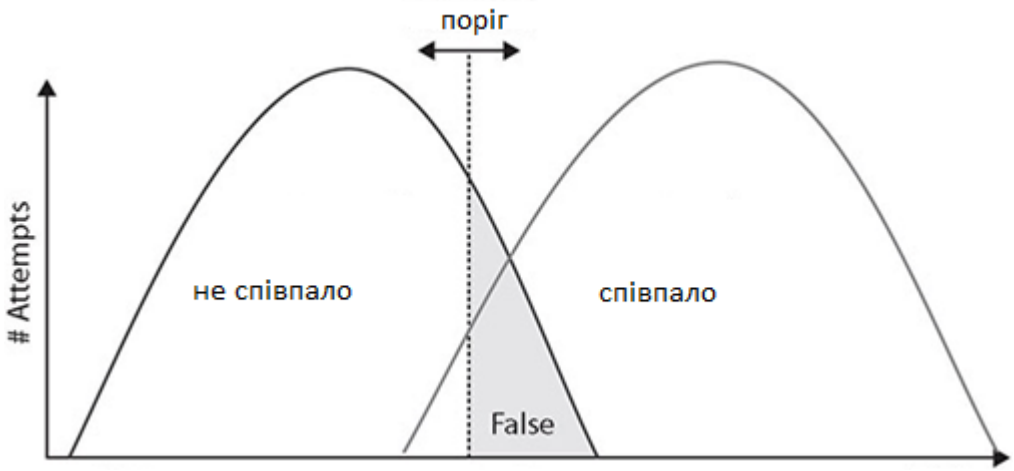
► **FAR** ( False Acceptance Rate ) - також іменується «**помилкою 2 роду**» . Величина **1- FAR** називається **специфічність** .

► **FRR** ( False Rejection Rate ) - також іменується «**помилкою 1 роду**». Величина **1- FRR** називається **чутливість**



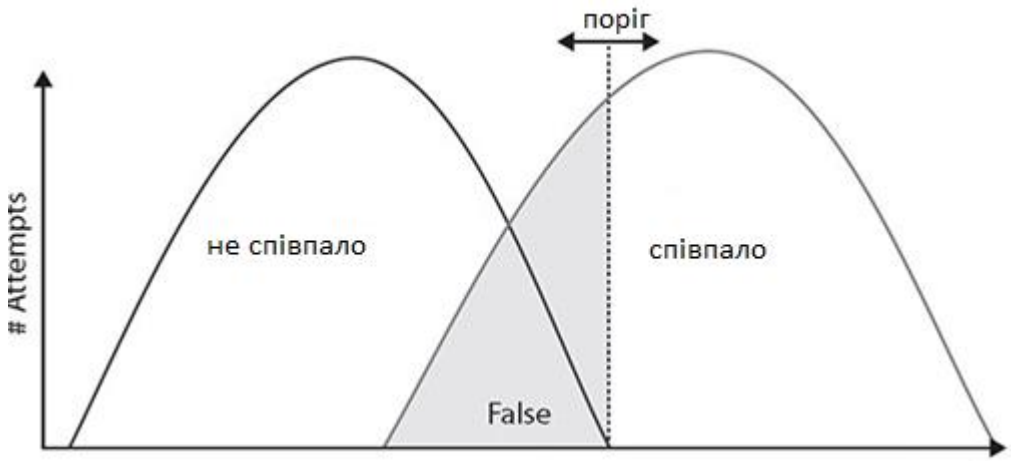

**Equal Error Rate (EER) : FRR = FAR**

# біометрична аутентифікація




Збільшуємо False acceptance rate

**Збільшення ризику**




Збільшуємо False rejection rate

Більш безпечно

# біометрична аутентифікація

- Коли ви оцінюєте біометричні системи, знання їх FAR та FRR допоможе вам визначити їх ефективність або наскільки вони ефективні у виконанні бажаної функції.
- Альянс FIDO встановлює поріг **FRR** для прийняття для сертифікації біометричних факторів **3** на **100** спроб (**3%**) та **1 на 10000 для FAR** (**0,01%)**.
- Вони також додають ще одну метрику - показник збігу атаки самозванця (**IAPMR - Imposter Attack Presentation Match Rate**), показник, який вирішує питання про те, як часто атака буде успішною. IAPMR є складним заходом, оскільки вимагає атак, розроблених спеціально для того, щоб скористатися слабкими сторонами будь-якої даної біометричної системи.
- Необхідно оцінити прийняття користувачем біометричної системи. Сканери сітківки не популярні, оскільки більшість людей не хочуть нахилятися і вдивлятися в сканер сітківки. Люди, які стерли свої відбитки пальців ручною працею або через хімічний чи інший вплив, не можуть використовувати багато пристроїв для зчитування відбитків пальців.

# біометрична аутентифікація

## Загальні характеристики методу

► **Нездатність зареєструватися** **Failure to enroll** (**FTE**): це кількість або відсоток осіб, які не можуть зареєструватися з тієї чи іншої причини. Причини - система не могла отримати гідний зразок або достатньо даних для створення шаблону.

► **Неможливість виділити характеристику** **Failure to acquire** (**FTA**): це кількість випадків, коли система не змогла виділити характеристику в процесі вимірювання. Причини включають той факт, що система не змогла отримати достатньо даних, таких як деталі відбитків пальців, або вибірка була замалою..

# біометрична аутентифікація

**Біометричні фактори**

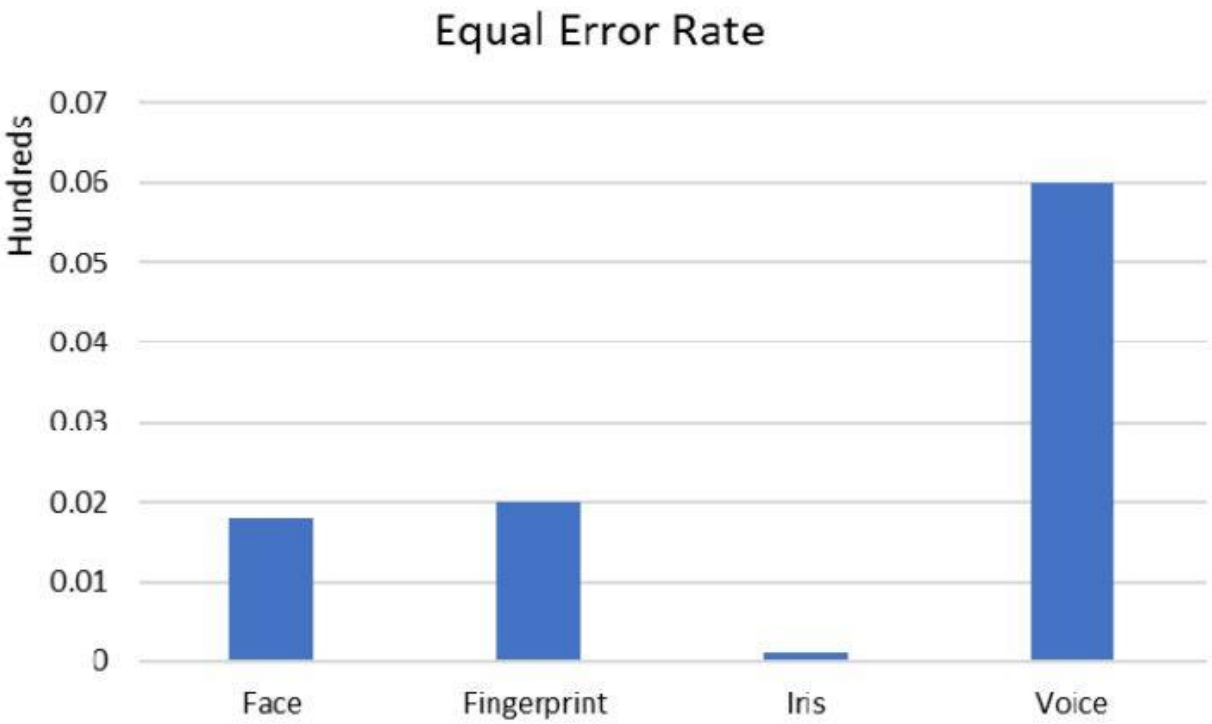
**Фізіологічні (Статичні)**

- відбиток пальця
- форма обличчя
- форма долоні
- сітківка ока
- райдужка

**Поведінкові (динамічні)**

- голос
- Підпис
- Хода
- Клавіатурній почерк

# біометрична автентифікація



# Криптографічні протоколи автентифікації

# Протокол автентифікації Kerberos

► Є незахищена мережа, в вузлах якої зосереджені **суб'єкти** - користувачі, а також клієнтські і серверні програмні системи.

► Кожен суб'єкт має секретний ключ.

► Щоб суб'єкт **A** міг доказати свою справжність суб'єкту **B** (без цього **B** не стане обслуговувати **A** ), він повинен не тільки назвати себе, але і продемонструє знання секретного ключа.

► **A** не може просто послати **B** свій секретний ключ, **по-перше**, тому, що мережа відкрита (доступна для пасивного і активного прослуховування), а, **по-друге**, тому, що **B** не знає (і не повинен знати) секретний ключ **B** . Потрібно менше прямолінійний спосіб показати знання секретного ключа.

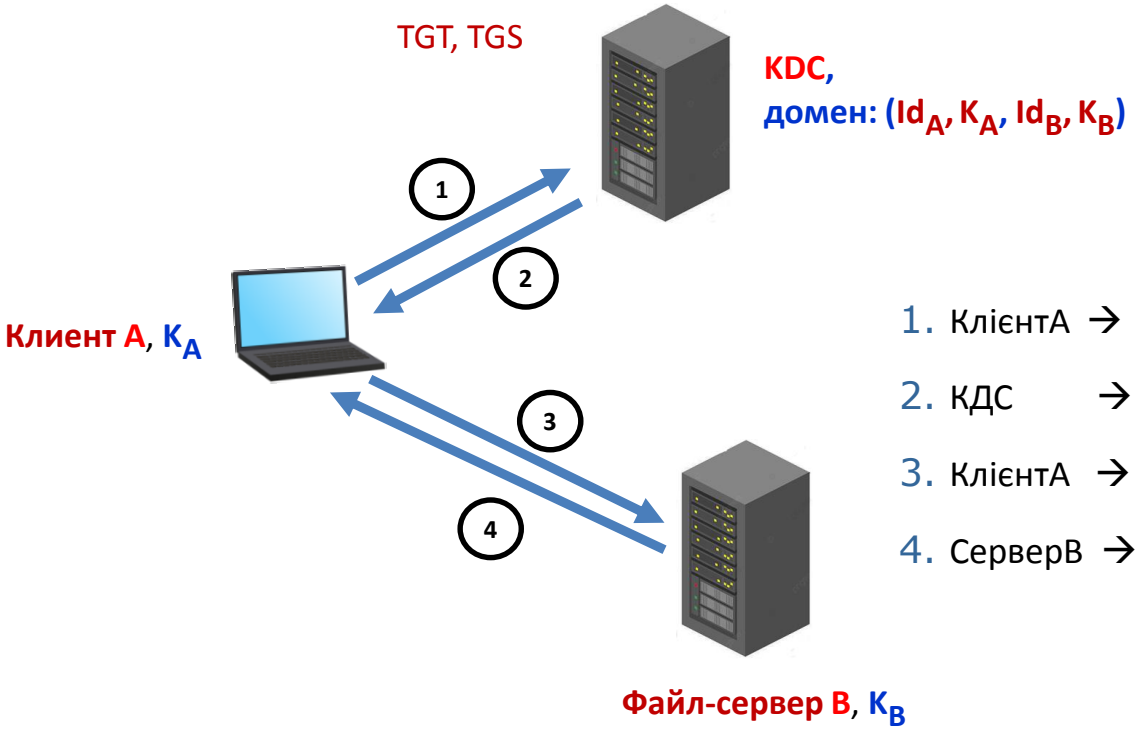
► **Ticket** є зашифрованим пакетом даних, виданими виділеним довіреним центром аутентифікації Key Distribution Center (**KDC** - центр розподілу ключів).

**Ticket Granting Ticket (TGT)**

**Ticket Granting Service (TGS)**

# Протокол автентифікації Kerberos

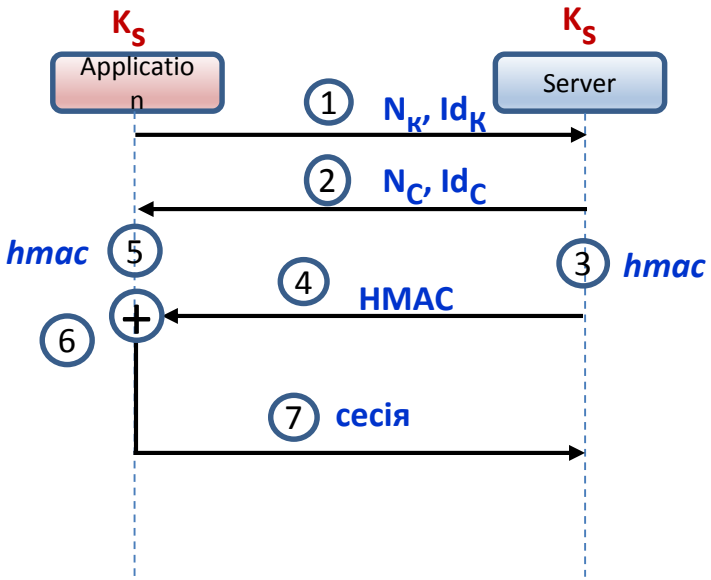
**Ka, Kb – ключи клієнта і сервера**




TGS

1. КлієнтА → КДС $Id_A$, $Id_B$, T
2. КДС → КлієнтА $[K_S, Id_B, T, \overbrace{[Id_A, K_S]K_B}^{TGS}]K_A$
3. КлієнтА → СерверВ $[Id_A, K_S]K_B$ $[Id_A, T]K_S$
4. СерверВ → КлиентА $[T]K_S$

# Коди автентифікації повідомлень ( HMAC)




**Nк, Nc** - 128 бітове число, вибране випадковим чином клієнтом і сервером окремо;
**$Id_K$, $Id_C$** - ідентифікатори клієнта і сервера (наприклад, IP адреса);
**HMAC** - хеш ідентифікації повідомлення;
**Ks** - загальний секретний ключ.

1. Клієнт посилає випадково вибране число **Nк** серверу
2. Сервер відправляє клієнту випадкове число **Nc** ,
3. Сервер формує хеш HMAC, утворений шляхом хешування значень **Nк** , **Nc, $Id_K$, $Id_C$** і **Ks** за допомогою HMAC-функції.
4. Север відправляє HMAC клієнту
5. Оскільки клієнт знає **Nк, Nc, $Id_K$, $Id_C$** і **Ks** , то він може формувати свій хеш HMAC,
6. Клієнт порівнює сформований HMAC з відповіддю сервера.
7. Якщо хеші будуть збігатися, то сервер є тим, за кого себе видає.

.

► Зловмисник не міг би сформувати хеш **HMAC (Nк, Nc, $Id_K$, $Id_C$, Ks)**, тому що він як мінімум не повинен знати секретного ключа **Ks**.

► Тепер прийшла черга клієнта підтвердити свою «особистість». Процес здійснюється за аналогією

# Коди автентифікації повідомлень ( HMAC)

**Nк, Nc - HOTP**

$$HMAC(K,P) = H(K \oplus opaq \,||\, H(K \oplus ipaq)||P) \quad \text{для } H = SHA\text{-}256$$

$\oplus$ - оператор XOR, а || - оператор конкатенації

**K –** секретний ключ
**P –** повідомлення **{Nк, Nc, $Id_K$, $Id_C$};**
**opad** - це зовнішнє заповнення, що складається з 0x5C (01011100), яке повторюється до довжини блоку хешування B,

**ipad** - це внутрішнє заповнення, що складається з 0x36 (00110110), яке повторюється до довжини B.

заповнення служить для того, щоб перемішувати половину бітів **K**

**Message = {Nк, Nc, $Id_K$, $Id_C$}; Ks =** 'thisIsASecretKey '

macCode = sha256('thisIsASe' + sha256('cretKey' + ' **Message** ')) $\Rightarrow$ **HOTP**

# Автентифікація за допомогою інфраструктури відкритих ключів ( PKI )

- **Public Key Infrastructure - PKI** - інфраструктура, призначена для управління **сертифікатами** відкритих ключів з метою підтримки послуг автентифікації, шифрування, цілісності і неспростовності (неможливість відказу від авторства).
- **Справжність** відкритого криптографічного ключа і його зв'язок з певним користувачем у рамках PKI засвідчуються **сертифікатом відкритого ключа**. Приналежність сертифіката відкритого ключа завіряється спеціальним довіреною установою - **центром сертифікації**.
- **Сертифікат відкритого ключа** - електронний документ, що містить відкритий ключ, інформацію про власника ключа, області застосування ключа, підписаний органом, що його засвідчує і підтверджує приналежність відкритого ключа власнику.

# Структура сертифіката

- ► унікальний реєстраційний номер сертифіката
- ► дати і час початку та закінчення терміну дії сертифіката
- ► Ідентифікатор (прізвище, ім'я та по батькові власника сертифіката ключа підпису або псевдонім) власника
- ► відкритий ключ власника
- ► реквізити ЦС
- ► найменування криптографічного алгоритму
- ► інформацію про обмеження використання підпису
- ► Цифровий підпис всіх полів сертифіката (хеш полів) створений закритим ключем ЦС

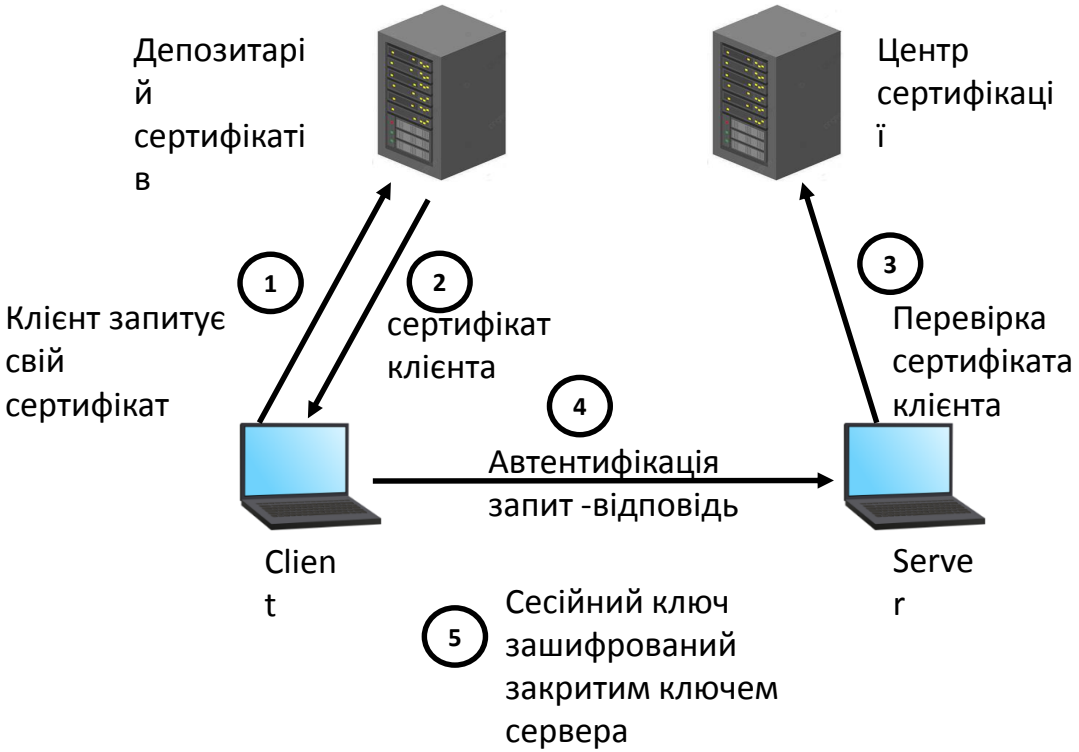
# Формат файлів сертифіката

| Стандарт | Формат | File extension | Comment |
|---|---|---|---|
| Distinguished Encoding Rules (DER) | Binary | .der, .crt, .cer | Найбільш популярний бінарний формат |
| Privacy Enhanced Mail (PEM) | Text | .pem, .crt | Текстова версія формату DER |
| Personal Information Exchange (PFX) | Binary | .pfx, .p12 | Зазвичай використовується в системах Windows |
| P7B | Text | .p7b | Використовується в системах Windows. Текстова версія формату PFX |

# Формат сертифіката

| | |
|---|---|
| Version | найменування криптографічного алгоритму |
| Serial number | Унікальний реєстраційний номер сертифіката |
| Signature | Цифровий підпис всіх полів сертифіката (хеш полів) створений закритим ключем ЦС |
| Issuer | Реквізити ЦС |
| Validity | дати і час початку та закінчення терміну дії сертифіката |
| Subject | Прізвище, ім'я та по батькові власника сертифіката ключа підпису або псевдонім власника |
| Subject public key info | Відкритий ключ власника |
| Issuer unique ID | Ідентифікатор ЦС |
| Subject unique ID | Ідентифікатор власника |
| Extensions | інформацію про обмеження використання підпису |

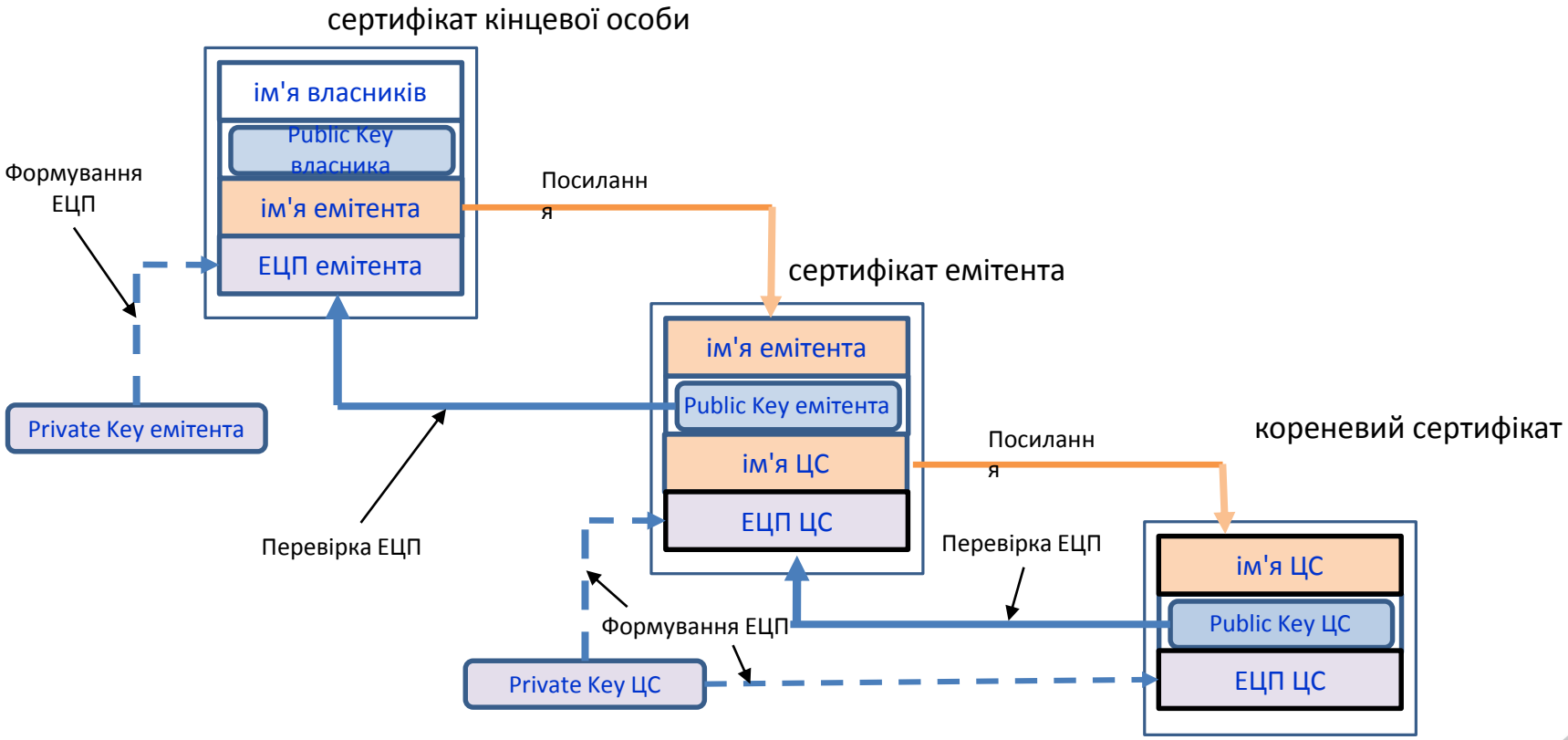
# Автентифікація з використанням сертифікатів відкритого ключа




Перевірка (3):

- ідентифікатора користувача:
- терміну дії сертифікату;
- дійсності відкритого ключа СА;
- підпис під сертифікатом користувача за допомогою відкритого ключа СА;
- сертифікат на предмет відкликання;

# Ланцюжок довіри сертифікатів



# OpenID Connect

► **OpenID** - відкритий стандарт децентралізованої системи автентифікації, що надає користувачеві можливість створити **єдину обліковий запис** для автентифікації на безлічі не пов'язаних один з одним інтернет-ресурсів, використовуючи послуги третіх осіб.

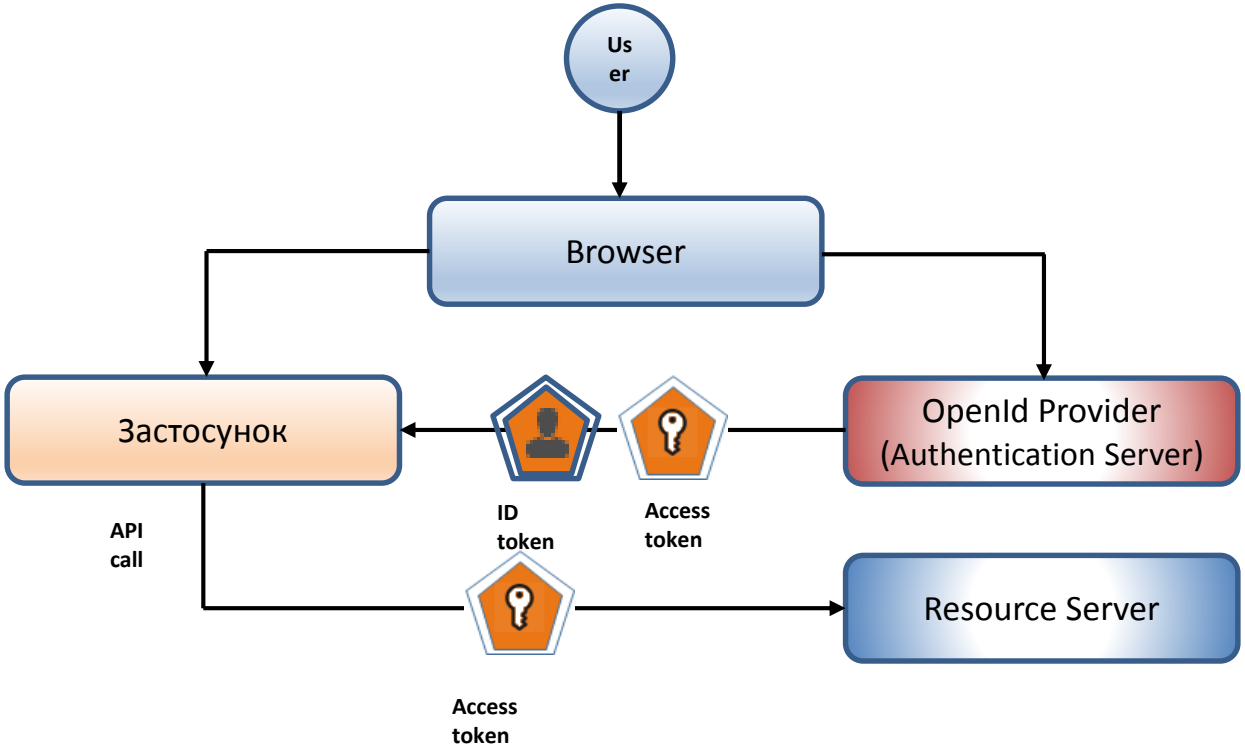
► Базовою функцією OpenID є надання портативного, клієнт-орієнтованого, **цифрового ідентифікатора** (**OpenID**) для вільного і децентралізованого використання

► Стандарт описує процес комунікації інтернет-ресурсів (Relying Parties - довіряючої сторони (RP), що вимагають автентифікації, і провайдерів OpenID (OpenID Providers).

► Існує кілька OpenID-провайдерів, які надають хостинг OpenID URL. Типовим прикладом цього є функція "**Увійти за допомогою Google**", яку надають багато веб-сайти, Microsoft, Amazon та багато інших організацій є **постачальниками ідентифікаторів OpenID (IdP).**

Довіряючі сторони (RP) перенаправляють запити автентифікації на **IdP**, а потім отримують відповідь із твердженням, що користувач є тим, кого вони претендують на успішну автентифікацію, і користувач входить в систему за допомогою OpenID для цього користувача.

# OpenID Connect




OpenID дозволяє користувачеві використовувати один обліковий запис, зареєстрований у OpenID-провайдера, на багатьох інших сайтах. Користувач може вибрати, яку інформацію надати сайту.

Для работы OpenID не требуется использовать cookie или какие-либо другие механизмы управления сессией

# ID Token

Алгоритм:
HS256 (HMAC with SHA256)
або
RS256 (RSA Signature with SHA256)

**iss** Емітент токена ID, визначений в форматі UrL. Емітент, як правило, є провайдер OpenID.

**sub** Унікальний (в межах провайдера OpenID), рядковий ідентифікатор автентифікованого користувача, довжиною до 255 символів.

**aud** Ідентифікатор клієнта перевіряючої сторони (застосунку), для якої призначений токен.

**exp** Час закінчення терміну дії маркера ідентифікатора, зазначений як кількість секунд від 1 січня 1970 року

Header (algorithm and type of token)
{“alg” : “RS256” ,
“Typ” : “JWT” }

Payload (claims)
{
“iss”: “http://openidprovider.com”,
"sub": "1234567890",
“aud” : “2fb3JsPMrDnQkwLEVNMDzUF”,
“nonce”:”47jglw0hmxo2hg0ewhg9582lf”,
“exp”: 1516239322,
"iat": 1516239022,
"name": ”Fred Doe",
"admin": true,
“auth_time”:1516239021,
“acr”:”1”,
“amr” : “pwd”
}

Signature

**nonce** Випадкове рядкове значення.

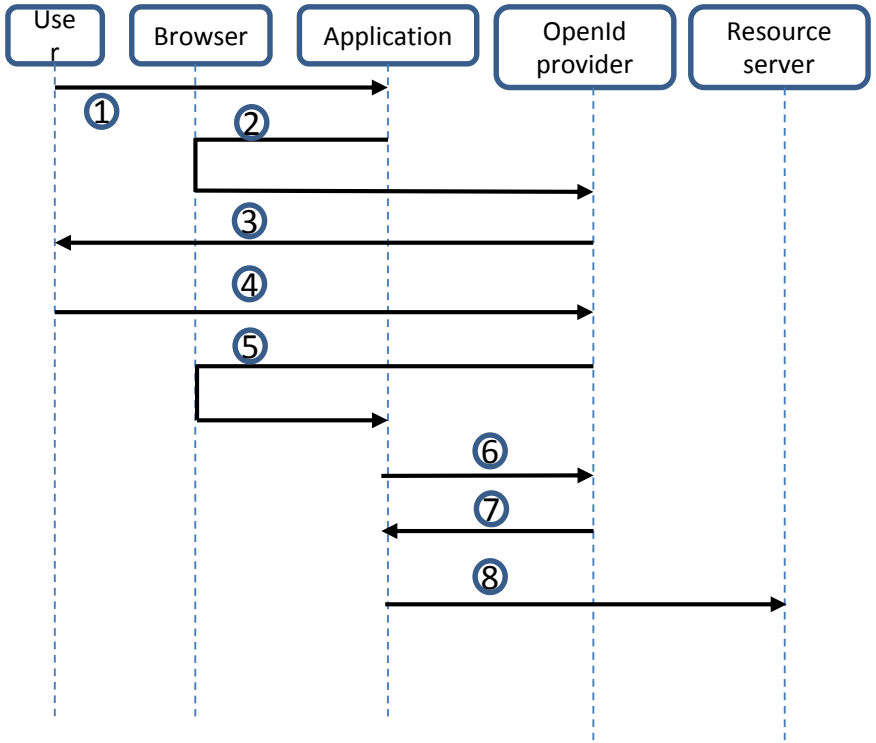
**iat** час випуску токена (кількість секунд, з 1 січня 1970)

**auth_time** час, коли користувач був автентифікований.

**acr** Рядок, який містить посилання на клас контексту аутентифікації.

**amr** Рядок, що містить посилання на метод автентифікації

# OpenID Connect




1. Користувач ініціює процес автентифікації в застосунку. Для цього він вводить ідентифікатор у форму входу, представлену на сайті. З ідентифікатора застосунок визначає URL OpenID-провайдера.
2. Застосунок направляє браузер користувача до OpenID Provider із запитом автентифікації. Запит містить дані автентифікації застосунку в OpenID Provider
3. OpenID Provider перевіряє, чи авторизований користувач на сервері і чи хоче він автентифікуватися в застосунку.
4. Користувач автентифікується (login) та дає згоду на надання інформації щодо облікового запису, а постачальник OpenID створює сеанс автентифікації для користувача.
5. Провайдер перенаправляє браузер користувача назад до застосунку, передаючи результат автентифікації. Застосунок перевіряє справжність інформації, отриманої від провайдера, включаючи повернутий URL, інформацію про користувача, одноразову мітку (nonce) та підпис повідомлення.
6. Застосунок надсилає провайдеру додатковий запит для автентифікації. У разі успішної перевірки застосунок автентифікує користувача.
7. Постачальник OpenID відповідає ID Token + маркером доступу (Access Token).
8. Застосунок може використовувати маркер для доступу до серверу ресурсів

# Види автентифікації

| Спосіб автентифікації | Фактор | Рівень автентифікації |
|---|---|---|
| Пароль, PIN | знання | проста |
| Пристрій OTP | володіння пристроєм | |
| Використання другого каналу (телефон + SMS) | володіння телефоном | |
| Пароль + пристрій OTP | володіння пристроєм + знання паролю (або біо для доступу к пристою) | посилена |
| Пристрій з встановленою криптографіей (ОТ або смартфон) | володіння ОТ+ знання паролю | |
| Засіб ОТ з криптографією (Kerberos) + доступ к ключу по паролю | володіння ОТ+ знання паролю | строга |
| Засіб ОТ з криптографією + зовнішній пристрій з криптографією (закритий ключ в чипі) | володіння пристроєм + знання PIN | |

# Автентифікація в WEB-застосунках

# Базова (base, HTTP ) автентифікація в ВЕБ - додатках

- підтримуваний всіма браузерами.
- ресурси, для доступу до яких необхідно пройти аутентифікації, поміщаються в окремий каталог, і створюється файл конфігурації, що містить набір обмежень.
- браузер відправляє HTTP запит на доступ до захищеного ресурсу

```
GET /dir/index.html HTTP/1.1
```

- серверний HTTP процес визначає, що необхідний доступ до захищеного ресурсу і зчитує з файлу конфігурації параметри (**ім'я домену**). З'ясувавши, що призначений для користувача запит не містить автентифікуючей інформації повертає відповідь з кодом **401**:

**WWW-Authenticate: Basic realm= "wwwProtectedSite“**

- Браузер, отримавши відмову, перевірять, чи немає в кеші пароля і логіна для зазначеної області даного сервера. Якщо не знаходить, то виводить діалогове вікно для введення імені і пароля. Введені користувачем дані **зберігаються** для подальшого використання в **кеші браузера**.
- Браузер відправить сервере відповідь з ознакою "**Authorization**" і тег, який містить значення "**Basic**" і конкатенації **імені, двокрапки й пароля** в кодуванні **base 64** . наприклад:

**Authorization: Basic QWRtaW46Zm9vYmFy.**

# Базова (base, HTTP ) автентифікація в ВЕБ - додатках

- серверний HTTP процес звіряє отриману інформацію з тієї, яка зберігається в базі облікових записів. Якщо обліковий запис з таким ім'ям не існує, або переданий пароль не збігається з тим, який зберігається в базі, сервер відкидає запит.
- Отримавши відмову, браузер повторно виводити вікно для введення імені або пароля.
- Якщо всі перевірки виявилися успішними, сервер пересилає браузеру потрібну HTTP сторінку.

**Недоліки**

- пересилання пароля практично у відкритому вигляді ( Base64)
- встановлене довірче з'єднання може бути розірвано тільки шляхом закриття браузера. Навіть якщо користувач перемкнув браузер на сайт, який не потребує автентифікації, ім'я та пароль зберігаються в кеші

# Автентифікація на основі хешу (digest)

- При спробі доступу браузера до захищеного ресурсу

```
GET /dir/index.html HTTP/1.1
```

- в повідомленні сервера вказується, що потрібно автентифікація на основі хешу і випадкове міститься значення ( **nonce** ), індивідуальне для кожного запиту.

```
HTTP/1.0 401 Unauthorized
Server: HTTPd/0.9
Date: Sun, 10 Apr 2020 20:26:47 GMT
WWW-Authenticate: Digest realm="testrealm@host.com", qop="auth,auth-int",
                         nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",
                         opaque="5ccc069c403ebaf9f0171e9517f40e41"
```

- браузер отримує пароль користувача (з діалогового вікна або з пам'яті)
- **opaque** - рядок даних, визначений сервером, який СЛІД повертати клієнтові без змін у полі заголовка авторизації наступних запитів з URI в тому самому просторі захисту. РЕКОМЕНДОВАНО, щоб цей рядок був в Base64 або шістнадцятковими даними

# Аутентифікація на основі хешу (digest)

- Об'єднує в один рядок ім'я користувача, ім'я області автентифікації і пароль, потім обчислює хеш отриманого значення

$$H_1 = MD5\ (Username: Realm: Password)$$

- Об'єднує в один рядок запитуваний URL з назвою методу, використовуваного в запиті ( **PUT , GET** , ...) і обчислює хеш отриманого рядка;

$$H_2 = MD5\ (method: URL)$$

- Виконує конкатенацію $H_1$ і $H_2$ зі значенням **nonce** і обчислює результуючий хеш.

$$Response = MD5\ (H_{1:} : nonce:\ H_2)$$

- Отримане значення хешу відправляється сервера

```
GET /dir/index.html HTTP/1.0
Host: localhost
Authorization: Digest username="Test",
realm="testrealm@host.com",
nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",
uri="/dir/index.html",
response="6629fae49393a05397450978507c4ef1",
opaque="5ccc069c403ebaf9f0171e9517f40e41"
```

# Аутентифікація на основі хешу (digest)

- Сервер отримує клієнтський запит, витягує зі сховища пароль користувача, на його основі повторює обчислення контрольних значень хеш-кодування. Можна зберігати не пароль користувача, а обчислене значення $H_1$ . Процедура автентифікації вважається успішною, якщо результат обчислень збігаються з тим, що отримано від клієнта.
- **.htdigest** - це файл, що використовується для зберігання імен користувачів, домену та паролів для дайджест-аутентифікації сервера **Apache**.

Ім'я файлу вказано в файлі конфігурації **.htaccess**.

Формат файлу **.htdigest:**

```
user1: Realm : 5ea41921c65387d904834f8403185412
user2: Realm : 734418f1e487083dc153890208b79379
```

# Аутентифікація на основі хешу (digest)

**Недоліки**

► Якщо трафік не шифрується за допомогою SSL або HTTPS, то атакуючий може переглядати всі запити в **текстовому вигляді**. Він може зберегти текст запиту цілком і пізніше повторити його.

► Сервер обмежений в можливості захищеного зберігання паролів. Оскільки для обчислення хешу використовується пароль, сервер повинен мати доступ до паролю у відкритому вигляді, або він повинен зберігати обчислене значення *хешу* . Таким чином, даний протокол не можна використовувати, якщо паролі захищені за допомогою **незворотного** шифрування.

► Якщо неавторизований користувач отримує доступ до бази паролів (хешів), йому немає необхідності знати пароль, він може скористатися значенням $H_1$ для атаки даного сайту.

► Дайджест-аутентифікація вразлива до атак **"людини посередині" (MitM**). Наприклад, MitM-зловмисник може сказати клієнтам використовувати базову автентифікацію або режим дайджест-автентифікації. У більш широкому змісті, **дайджест-автентифікація доступу не надає клієнтам механізмів перевірки сервера**.

# Автентифікація на основі форм

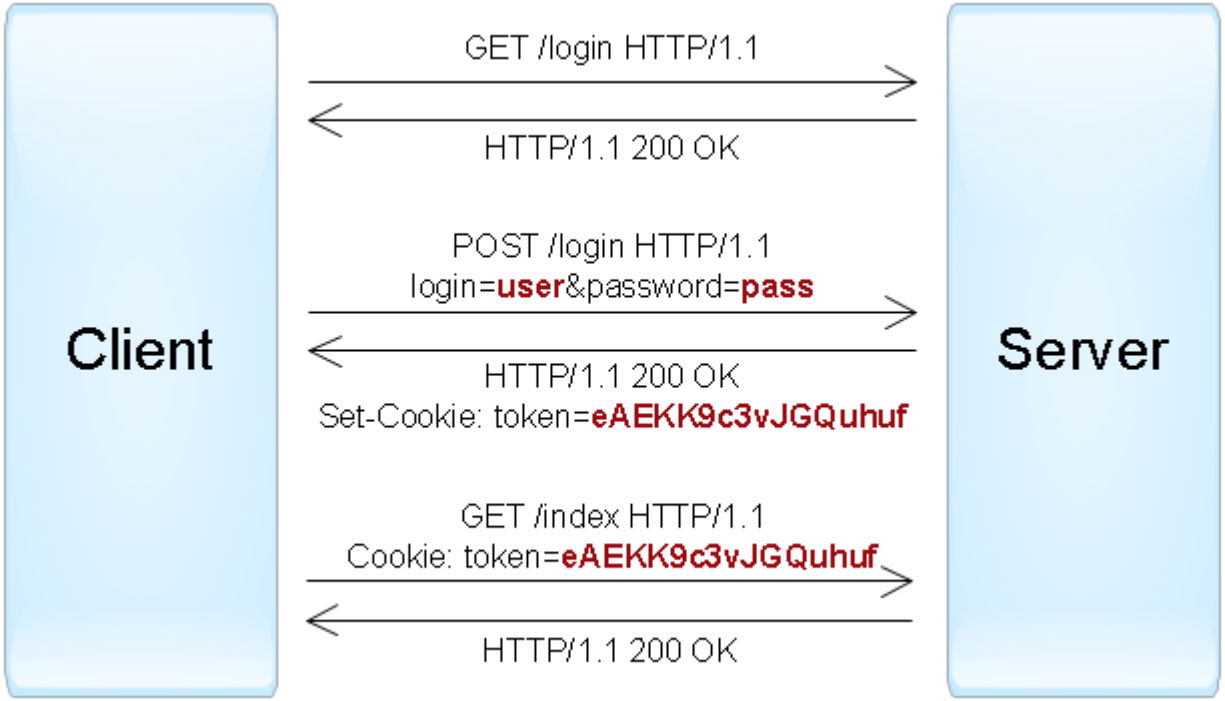
Форма - це **основний спосіб** введення інформації на стороні клієнта, для відправки її серверу (точніше, Web -додатків).

Розробники часто використовують процедури автентифікації власної розробки, намагаючись зробити їх більш захищеними за рахунок ускладнення . Застосовуються, зокрема, такі заходи:

- Вимога до користувачів **більш повно ідентифікувати** себе, вказуючи не лише ім'я та пароль;
- Використання захисту від атак **підбору пароля** (brute force);
- Запровадження механізму **управління** призначеними для користувача **сесіями**, зокрема, їх завершення через певний час;
- Для **балансуванням навантаження** на сайт, використовується кешування на стороні клієнта або проксі-сервіси.

**Стандартної реалізації даного методу автентифікації немає.**

# Автентифікація з використанням cookie




- **Максимальний розмір** кожного файлу cookie (ім'я та значення разом) становить **4 КБ**. Допускається максимум **20 файлів cookie на сервер** або доме
- При автентифікації на основі форм необхідно забезпечити **захист даних автентифікації що передаються**, так як їх можна викрасти за допомогою програми моніторингу мережі. Треба захищати з'єднання за допомогою **SSL/TLS**.

# Автентифікація на основі cookie

► автентифікація з використанням cookie може бути задана в такий спосіб у файлі конфігурації (наприклад **Web.Config** для ASP.NET web-застосунків):

```
<authentication mode="Forms">
<forms name="MyCookieName" loginUrl="LogPage.aspx"
timeout="20"/>
</authentication>
```

► В елементі **`form`** атрибуту **`loginUrl`** задається посилання на web-форму, до якої потрібно буде перенаправити запит для автентифікації користувача (в даному випадку це web-форма **`LogPage.aspx`**),

► атрибут **`timeout`** задає час між запитами (в хвилинах), після якого користувачеві буде потрібно заново виконати автентифікацію.

# Структура Cookie

| domain | flag | path | secure | термін дії | name | значення |
|---|---|---|---|---|---|---|
| www.redhat.com | FALSE | / | FALSE | 1154029490 | Apache | 64.3.40.151.16018996349247480 |

- **name**: ім'я змінної (в даному випадку Apache).
- **domain**: Домен веб-сайту, який створив і може читати cookie.
- **flag**: значення TRUE / FALSE, яке вказує, чи всі машини в даному домені можуть отримати доступ до cookie .
- **path**: атрибут path надає діапазон URL-адрес, для якого файл cookie дійсний. Якщо шлях встановлений на **/reference**, cookie буде відправлений для URL-адрес в **/reference**, а також для підкаталогів. Шлях "**/**" вказує, що файл cookie буде використовуватися для всіх URL-адрес на сайті, з якого був створений файл cookie.
- **secure**: значення TRUE / FALSE, яке вказує, чи потрібно SSL-з'єднання з доменом для доступу до ресурсу.

# Структура Cookie

| domain | flag | path | secure | термін дії | name | значення |
|---|---|---|---|---|---|---|
| **www.redhat.com** | FALSE | / | FALSE | 1154029490 | Apache | 64.3.40.151.16018996349247480 |

► **термін дії (Expires)**: час Unix, після закінчення якого закінчується термін дії змінної. Час Unix визначається як кількість секунд, що пройшли з 00:00:00 за Гринвічем 1 січня 1970 року. Коли встановлюється термін дії, час і дата встановлюються не відносно сервера, а відносно клієнта, на якому встановлено cookie

► **Max-Age=<number>** : кількість секунд, після чого cookie застаріває. Нуль або негативне число призводять до моментального старіння cookie. Якщо обидва параметри (**Expires** і **Max-Age**) встановлені, Max-Age буде мати **перевагу**.

► **HttpOnly** Забороняє доступ к cookie з JavaScript .

► **SameSite = Strict**: Браузер надішле cookie тільки якщо запит отримано з того ж сайту (заголовок **Referer** в відповіді сервера**)**.

# Автентифікація з використанням cookie

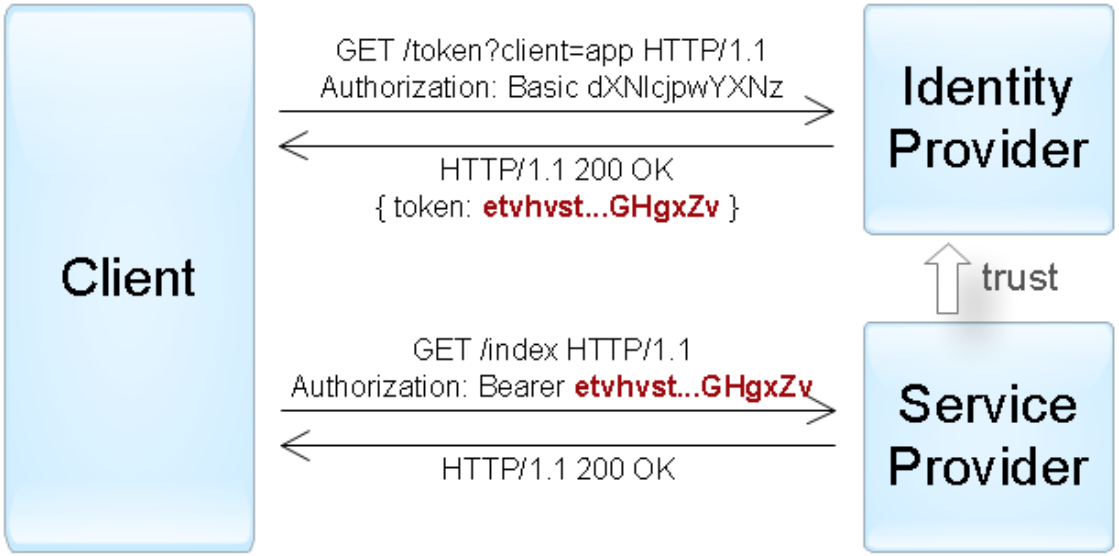
**Плюси**

- Швидший наступний вхід, оскільки облікові дані не потрібні.
- Досить легко реалізувати.
- Багато фреймворків (наприклад, Django) надають цю функцію «з коробки».

**Мінуси**

- Сервер відстежує кожну сесію. Щоб увімкнути автентифікацію, сховище сесій, яке використовується для зберігання інформації про сеанс користувача, має бути спільне між кількома службами. Через це він погано працює для служб RESTful, оскільки REST є протоколом без стану.
- Файли cookie надсилаються з кожним запитом, навіть якщо він не вимагає автентифікації
- Уразливий до атак CSRF

# Автентифікація активного клієнта за допомогою токенів




**Активний клієнт** – це натівний **iOS/Android/Windows** додаток або single page apps

1.Клієнт автентифікується в **identity provider** одним із способів, специфічним для нього (пароль, ключ доступу, сертифікат, Kerberos, і тд.).

2. Клієнт просить **identity provider** надати йому токен для конкретного **SP-додатку**. **Identity provider** генерує токен і відправляє його клієнту.

3.Клієнт **автентифікується в SP-додатку** за допомогою цього токена.

Найбільш часто використовуваним є JSON веб-токен (JWT).

# Автентифікація активного клієнта за допомогою токенів

**Плюси**

► stateless

► Серверу не потрібно зберігати токен, оскільки його можна перевірити за допомогою підпису. Це робить запит швидше, оскільки пошук у базі даних не потрібен.

► Підходить для архітектури мікросервісів, де декілька служб вимагають автентифікації. Все, що нам потрібно налаштувати на кожному кінці, це як поводитися з токенів і ключем токена.

**Мінуси**

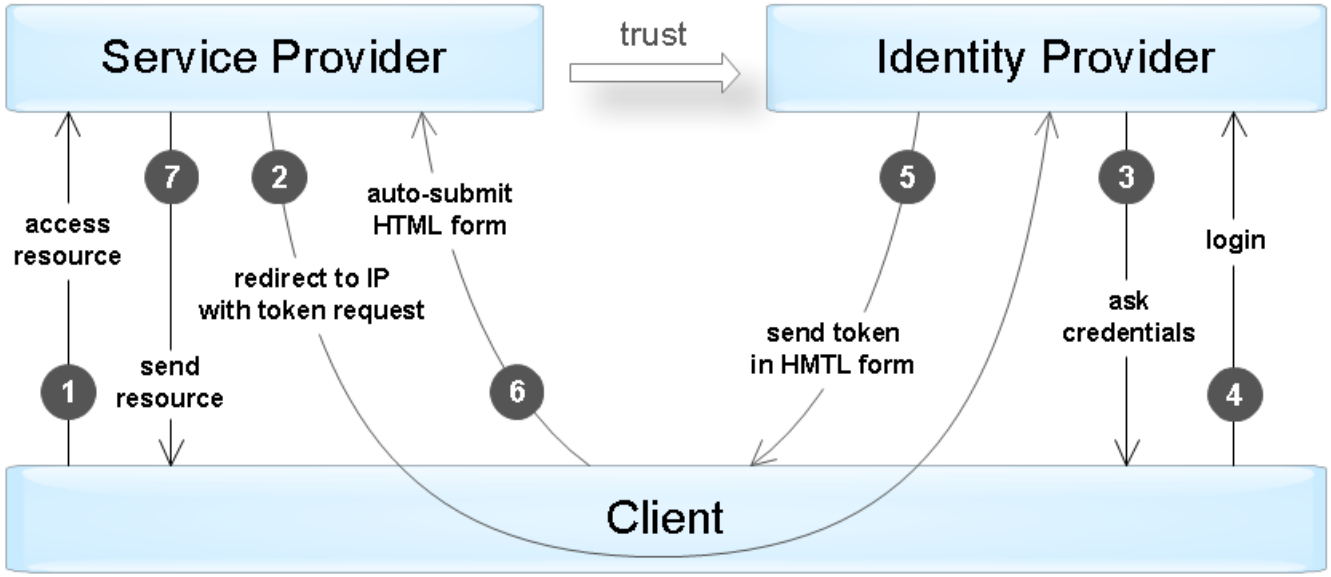
► Залежно від того, як токен зберігається на клієнті, це може призвести до атак XSS (через localStorage) або CSRF (через файли cookie).

► Токени не можна видалити. Вони можуть лише застаріти. Це означає, що в разі витоку токена зловмисник може використати його до закінчення терміну дії. Таким чином, важливо встановити термін дії маркера на щось дуже невелике, наприклад, 15 хвилин.

► Потрібно налаштувати оновлення токенів, щоб автоматично видавати токени після закінчення терміну дії.

► Одним із способів видалення токенів є створення чорного списку токенів. Це додає додаткові витрати на архітектуру мікросервісів і вводить стан.

# Автентифікація пасивного клієнта за допомогою токенів

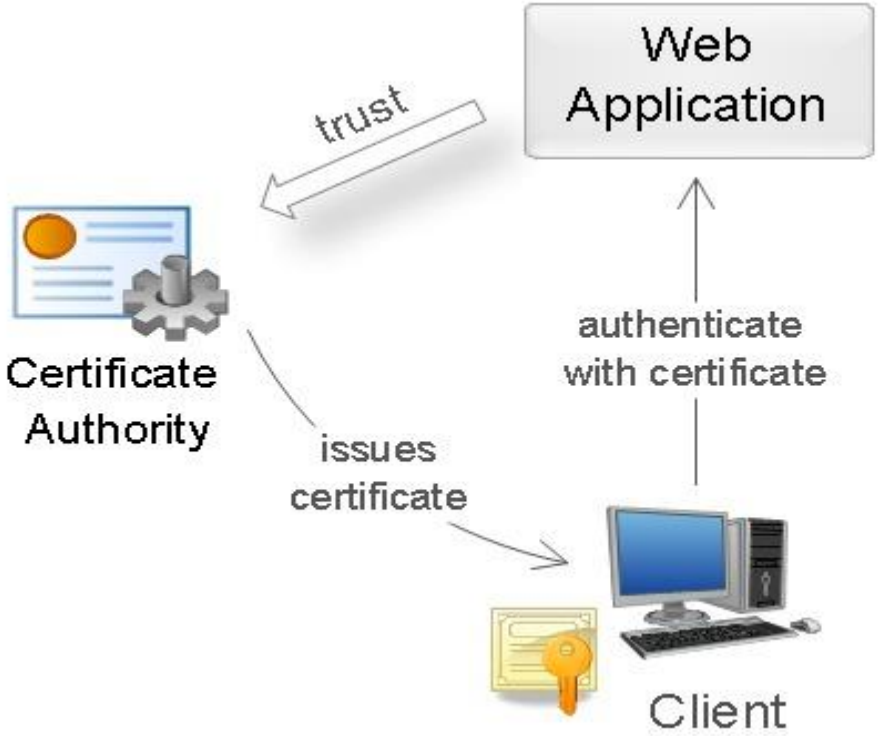



► **Браузер - пасивний клієнт** в тому сенсі, що він тільки може відображати сторінки, запитані користувачем.

► В цьому випадку автентифікація досягається за допомогою автоматичного перенаправлення браузера між **веб-додатками, identity provider і service provider**

► Популярний варіант автентифікації допомогою токенів – стандарт **OAuth**

# Автентифікація за допомогою сертифікатів




► У веб-додатках традиційно використовують сертифікати стандарту **X.509**. Автентифікація за допомогою X.509-сертифіката відбувається в момент з'єднання з сервером і **є частиною протоколу SSL / TLS**.

► На стороні **клієнта** сертифікат разом з закритим ключем може зберігатися в операційній системі, в браузері, в файлі, на окремому фізичному пристрої (smart card, USB token). Зазвичай закритий ключ додатково захищений паролем або PIN-кодом.

# Автентифікація за допомогою ключів доступу

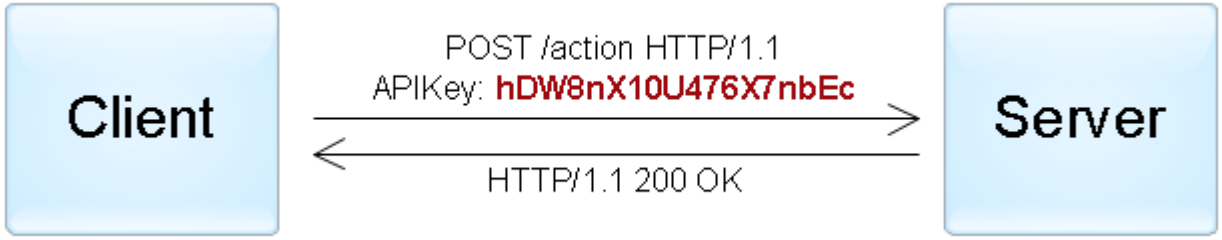
► Цей спосіб найчастіше використовується для автентифікації сервісів або **інших додатків** при зверненні до веб-сервісів. Тут в якості секрету застосовуються **ключі доступу** (access key, API key) - **довгі унікальні рядки**, що містять довільний набір символів, по суті замінюють собою комбінацію username / password.

► У більшості випадків, **сервер** генерує ключі доступу за запитом користувачів, які далі зберігають ці ключі в клієнтських додатках. При створенні ключа також можливо обмежити термін дії і рівень доступу, який отримає клієнтську програму при автентифікації за допомогою цього ключа.

► Наприклад, додаток може зберігати файли в хмарі Amazon Web Services

# Автентифікація за допомогою ключів доступу

► Використання ключів дозволяє уникнути передачі пароля користувача стороннім додаткам (в прикладі вище користувач зберіг в веб-додатку не свій пароль, а ключ доступу).
► Ключі мають значно більшу ентропією в порівнянні з паролями, тому їх практично неможливо підібрати.
► Крім того, якщо ключ був розкритий, це не призводить до компрометації основний облікового запису користувача - достатньо лише анулювати цей ключ і створити новий.
► З технічної точки зору, тут не існує єдиного протоколу: ключі можуть передаватися в різних частинах HTTP-запиту.
► Як і в випадку автентифікації по паролю, найбільш оптимальний варіант - використання **HTTP header**. Щоб уникнути перехоплення ключів, з'єднання з сервером має бути обов'язково захищене протоколом SSL / TLS.



# Установка довірчого з'єднання. Автентифікація в протоколі SSL

клієнт

ВЕБ сервер

1 Початкове з'єднання. URL ресурсу

Клієнт повідомляє який криптоалгоритм використовує

2 TLS/SSL handshake. Перевірка сертифіката

Сервер надсилає сертифікат відкритого ключа

3 Генерує і шифрує відкритим ключем «секрет» і надсилає серверу

4 Сервер і клієнт використовують секрет, генерують симетричний ключ і шифрують подальшу сесію

# Установка довірчого з'єднання. Автентифікація в протоколі SSL

**Проблеми**:

► Якщо веб-сервер не налаштований на використання SSL може бути встановлено звичайне **незахищене з'єднання**. Безпека залежить від того, вкаже користувач в рядку URL браузера протокол **https: /** замість **http: / (**атака SSL stripping**)**

► Якщо у клієнта немає копії сертифіката CA, вона буде запропонована йому сервером. Незважаючи на те, що це забезпечує наявність шифруемого з'єднання між клієнтом і сервером, даний підхід **не забезпечує автентифікацію сервера**.

► Процес отримання сертифікату CA в сховище браузера не є чітко контрольованим. Microsoft вимагає, щоб сертифікати, що встановлюються  на всі браузери, були випущені центрами сертифікації, які пройшли відповідний аудит.

# Установка довірчого з'єднання. Автентифікація в протоколі SSL

**Проблеми**:

► Основою є захист секретного ключа. За замовчуванням вимагають лише знаходження ключа в захищеній області системи.

► Як у випадку з будь-якою системою, що базується на PKI, можуть допускатися різні помилки Сертифікат SSL, який визначає сервер як такий що належить компанії, може бути випущений будь-якою особою, яка не є представником цієї компанії.

► Якщо навіть закінчився термін дії сертифіката або виявлена інша проблема і виданий сигнал тривоги, багато користувачів просто проігнорують це попередження.

# Блокування та контроль облікового запису

► **Послідовне збільшення тимчасового інтервалу** між моментом невдалої спроби автентифікації і відправкою запиту на повторну автентифікацію (зазвичай в два рази). Крім цього, необхідно обмежувати максимально допустиме значення інтервалу затримки і передбачити **скидання** лічильника наприклад, **раз на добу**.

► Якщо обліковий запис автоматично блокується після певної кількості невдалих спроб, то клієнт **не повинен знати причину** блокування. Якщо ніяких спроб аутентифікації не буде зроблено протягом заданого проміжку часу (наприклад, 1 година), обліковий запис може бути автоматично розблокована.

► Якщо після того, як обліковий запис була заблокована, але до моменту її розблокування, буде вказана **правильна** автентифікаційна інформація, клієнту повинна бути надана інформація про те, що запис заблокований з інструкцією по її розблокування.

# Блокування та контроль облікового запису

.

► Для Web -сайтів, які потребують більш суворої автентифікації, ніж вказівку імені та пароля, необхідно організувати процедуру автентифікації в кілька етапів у вигляді **запит-відповідь** .

► У веб-додатках має бути реалізована можливість відслідковувати та реєструвати з'єднання за IP-адресою клієнта. За цим додатком слід визначити наявність невдалих спроб автентифікації різних облікових записів **з одного і того же IP-адреси** та запропонувати відповідні заходи. Якщо дозволять засоби захисту зовнішнього периметра (брандмауерів або роутерів), потрібно динамічно блокувати зовнішню IP-адресу.

► Додатково для користувачів, успішно пройдених процедур аутентифікації, можна запропонувати можливість перегляду **історії невдалих спроб** після останніх вдалих спроб. Така інформація може бути використана для додаткового захисту навчальних записів або для інформування користувача про те, що забезпечується захист від таких випадків.

# Захист при доступу клієнта з загальнодоступних комп'ютерів

Необхідно передбачити захист від таких загроз, **як троянські коні, клавіатурні шпигуни, атак перехоплення**

► **Зміна процедури введення пароля** . Наприклад, не вводити весь пароль, а вказати лише деякі його символи, номера яких сервер генерує випадковим чином

► кращим є використання методу **POST** для пересилання даних

► Що б гарантовано захиститися від запису інформації на комунікаційних серверах, у відповідь сервера необхідно включити такі ключові слова:

Pragma: no-cache Cache-Control: private, max-age = 0, no-cache Expires: 01/01/99 20:00:00 GMT

В розділ HEAD сторінки повинні бути включені наступні теги :

```
<meta http-equiv  = "expires" content = "01/01/99 20:00:00 GMT">
<meta http-equiv  = "pragma" content = "no-cache">
<meta http-equiv  = "cache -control "content =" max-age = 0 ">
<meta http-equiv  = "cache-control "content =" no-cache ">
<meta http-equiv  = "cache-control "content =" no-store " >
```

# Захист при доступу клієнта з загальнодоступних комп'ютерів

► Браузер клієнта запросить нову копію сторінки, якщо в поле " **Expires** " буде встановлена дата, що вже пройшла. Цей метод дозволяє захиститися від використання кнопки " back button " браузера.

► Сучасні браузери можуть запам'ятовувати дані введені в поля форми, та згодом можуть заповнювати їх автоматично. Необхідно переконатися, що розробники використовують скрипти для автоматичної **очистки полів введення** форми. Коли браузер клієнта відкриває сторінку, вбудований скрипт матиме очищати все поля введення форми